

Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Asynchronous and Event-Driven Distributed Systems

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Publish–subscribe systems decouple components in space and time, which also hides how failures propagate between them. We present Software-as-a-Graph (SaG), a pre-deployment analysis framework that turns architecture descriptions into typed multigraphs with Quality-of-Service (QoS) weighted dependencies and ranks components by their cascading-failure impact, without runtime data. SaG provides a closed-form engine, graph-learning engines with 16-dimensional QoS edge encodings, and a hybrid engine in which a heterogeneous graph transformer learns a correction to the closed-form score. An ISO/IEC 25010 attribution layer explains why a component is critical. Under leave-one-scenario-out cross-validation over twelve synthetic architectures, SaG’s QoS-aware projection improves closed-form ranking on every held-out architecture (Spearman correlation 0.553 vs. 0.349). The hybrid engine performs best, reaching 0.657 and outperforming closed-form ranking on 11 of 12 held-out architectures (+0.103, Holm-adjusted $p = 0.0068$), as registered before the run. A control matched in model capacity and edge-channel width shows that the QoS edge encoding, not relation-specific weights, drives learned accuracy. On hand-authored models of five open-source systems, learned engines transfer zero-shot at 0.76–0.81, against 0.51–0.53 for every training-free score, and roughly double top-K critical-set overlap. Neural inference takes milliseconds, and feature extraction dominates analysis cost. These results show that QoS-aware architecture models give pre-deployment dependability analysis a strong foundation, and that learned and closed-form reasoning are most effective in combination.

Keywords: Heterogeneous graph neural networks, dependability, publish–subscribe, cascading failures, static system analysis, quality of service, explainable AI

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems progressively use asynchronous, event-driven, publish–subscribe (pub-sub) architectures. These are common within autonomous driving (ROS 2 [1]), enterprise event streams (Apache Kafka [2]), cyber-physical systems (DDS [3]), IoT deployments (MQTT [4]), cloud-native microservices [5, 6], and distributed AI/LLM clusters. Pub-sub architectures separate producers and consumers in terms of space, time, and synchronization [7]. Components communicate indirectly through message topics and brokers, so they do not need direct static references. Current middleware also supports deployment-time Quality-of-Service (QoS) policies, such as reliability, durability, message priorities, and delivery deadlines, to help manage performance during peak loads and network stress.

While this separation allows for flexible scaling, it also makes it harder to see what is happening inside the system. Unlike synchronous architectures like RESTful HTTP or gRPC, which show clear caller and

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

callee paths, pub-sub publishers and subscribers lack direct links. As a result, failures, head-of-line blocking, and backpressure can spread along concealed paths involving brokers, shared topics, colocated hosts, and shared libraries [8, 9]. These failures mainly happen in two ways: sequential cascades, where a slow subscriber fills up a broker queue and gradually slows down its publishers [10]; and simultaneous blast radii, where a shared library crash or host outage brings down all colocated services. Standard architecture diagrams and static call graphs cannot show these processes. The best time to reduce these risks is before deployment, especially during design and continuous integration, following dependable computing principles [11, 12]. At these points, runtime telemetry, distributed tracing, and logs are not available. Therefore, architects and Site Reliability Engineers need to identify the most critical components, topics, and links, understand why they matter, and take targeted actions like broker replication, splitting overloaded topics, or isolating shared libraries to reduce risk. Here, systemic criticality means an entity is either a single point of failure that can disrupt downstream connections if it fails, or an error-propagation hub that can cause large cascading outages in the asynchronous message mesh.

Analyzing architecture directly from configuration manifests removes the need for staging clusters, active containers, and live fault-injection tools, and lets criticality be assessed at design and integration time. The question this paper answers is which representation and which analysis engine make that assessment accurate.

1.2. Problem Statement: The Architecture–Code Gap and the Black-Box AI Challenge

Pre-deployment dependability analysis in distributed architectures has two main tasks. The first is predictive forecasting: whether a data-driven, relation-specific graph neural network can predict cascading failure blast radii and identify critical components better than traditional topological metrics. While closed-form metrics show global network connectivity, it remains unclear whether they can track multi-hop, relation-dependent cascade spread across several channels. We test this directly against such a baseline (Section 7.1). We train and evaluate the predictive model using independent simulation ground truth to rank and identify critical sets.

Explainable Criticality Attribution (the Explanation Layer) addresses the limits of ranking alone. A ranked list shows where risk is highest, but not how to fix it. To solve this, the predictor is combined with an interpretable structural quality profile based on ISO/IEC 25010 [13] and ISO/IEC 25019 [14]. This layer identifies structural risk types using standard software quality models. For example, it can distinguish between an unreplicated single point of failure, an error-propagating cascade hub, and a maintainability bottleneck. This helps guide targeted architectural fixes. The layer is used only for attribution, not for ranking.

This separation is built into the architecture, not just the presentation: both pathways use the same graph but do not share parameters, and neither is trained on the other’s output. The term that could connect them is off by default and reported only as an ablation (Section 4.2). This independence lets Software-as-a-Graph (SaG) find components that are structurally central but have low operational impact, supporting a more detailed diagnosis than either pathway could provide alone.

The distance between an architecture as designed and as realized is long-established: Perry and Wolf [15] formulated architectural erosion three decades ago, and the architectural-technical-debt literature has tracked it since [16, 17]. What we term the **Architecture–Code Gap** specializes this concept to asynchronous middleware, where the challenge is that failure semantics were never expressible in artifacts a build pipeline can inspect: *a distributed system can have pristine, bug-free source code within each service, yet remain fragile to ruinous global outages caused by hidden single points of failure (SPOFs) or mismatched middleware Quality-of-Service (QoS) contracts*. This fragility is acute in pub-sub architectures where producers and consumers interact indirectly without static references. Classical architecture evaluations (e.g., ATAM [18, 12]) rely on manual elicitation; static code analysis [19, 20, 21, 22] inspects individual services in isolation; chaos engineering [23] requires provisioned staging clusters; and homogeneous centrality [24, 25, 26, 27] flattens typed topologies into untyped graphs. Section 2 develops each paradigm in detail.

Although machine learning has made big advances in software engineering, current AI methods for system dependability often work as black boxes. Deep neural models usually give risk scores or hidden representations lacking clear, actionable reasons for their predictions. In mission-critical software engineering, this shortage of

transparency is not enough. Developers and Site Reliability Engineers need clear explanations of component vulnerabilities and architectural problems so they can refactor code or adjust infrastructure effectively.

1.3. The Software-as-a-Graph (SaG) Approach

To address both the Architecture–Code Gap and the black-box AI problem, this work describes **Software-as-a-Graph (SaG)**, an AI-based pre-deployment **Static System Analysis (SSA)** framework for asynchronous and event-driven distributed systems. SaG provides the tools and experiments needed to determine whether static representations before deployment can identify systemically critical components and explain their systemic root causes. SaG uses a four-stage pipeline: (1) modeling the architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts, and Shared Libraries (Section 3.1); (2) mapping physical connections into a QoS-weighted semantic **DEPENDS_ON** layer that captures sequential cascades and simultaneous failures (Section 3.2); (3) training a Heterogeneous Graph Transformer (HGT) to predict multi-hop cascading blast radii (Section 4); and (4) evaluating an explainable Reliability–Maintainability (RM) profile (Section 5) to show why components are fragile.

Importantly, SaG enforces a strict **input–label independence guarantee**: the learned models and attribution baselines use only the analytical graph G_{analysis} . Meanwhile, ground-truth failure impacts come from independent discrete-event simulators running on the raw structural topology $G_{\text{structural}}$ (Section 4.4). This keeps feature construction and ground truth structurally separate. Section 3 explains the full architectural flow and pipeline interactions (see Figure 1).

Main findings. Across twelve synthetic architectures under leave-one-scenario-out (LOSO) cross-validation, and five independently authored models of open-source systems, SaG’s representation and engines deliver consistent gains over standard practice:

1. **QoS-aware dependency modeling pays off.** Ranking on SaG’s QoS-weighted **DEPENDS_ON** projection raises the Spearman correlation with simulated cascade impact from 0.349 (unweighted centrality) to 0.553, improving all twelve held-out architectures (+0.204, $p = 0.0005$; Section 7.1).
2. **The QoS edge channel is what learned engines need.** In a control that matches model capacity and edge-channel width, SaG’s 16-D QoS edge encoding improves learned ranking by about 0.07 on 10 of 12 folds, while relation-specific weights add nothing beyond it (Section 7.2). Learned engines with the QoS channel reach $\rho = 0.62$ – 0.64 out of distribution, statistically on par with SaG’s closed-form engine on their own (+0.085, $p = 0.151$ for HGT-QoS).
3. **Learned engines transfer to unseen systems.** Trained only on synthetic scenarios, learned engines rank components zero-shot on the five open-source system models at $\rho = 0.760$ (HGT-QoS) and 0.805 (capacity-matched untyped GAT with the QoS channel), against 0.51–0.53 for every training-free score, with non-overlapping intervals. They roughly double top- K critical-set overlap (0.47–0.52 against 0.248) and raise PR-AUC from 0.47–0.52 to 0.71–0.79 (Section 7.4).
4. **A hybrid engine significantly outperforms closed-form ranking.** SaG-Hybrid, which lets the learned engine correct SaG’s closed-form score rather than replace it, reaches $\rho = 0.657$ and beats the closed-form engine on 11 of 12 held-out architectures (+0.103, CI [+0.055, +0.152], Holm $p = 0.0068$), meeting a decision rule registered before the run (Section 7.5).

Why a learned engine. The simulation oracle that labels the training data is available at design time only when the manifest declares every operational parameter it needs. A trained engine instead scores a new architecture in 56 ms once its features exist (Section 7.6), generalizes to architectures outside its training distribution (Section 7.4), and can be combined with the closed-form engine (Section 7.5). Section 8.4 lists the uses of learning that this study does not yet evaluate.

1.4. Research Questions

This empirical study considers five research questions:

- **RQ1 (Predictive Efficacy):** *How accurately does heterogeneous graph learning predict cascading failure impact and identify the critical component set, compared with traditional, non-learning network indicators?*
- **RQ2 (Value of Architectural Typing):** *Does modeling distinct entity and dependency types yield better failure predictions than homogeneous graph models on architectures the model has never seen — and does whatever advantage it confers depend on what other relational signal the model already has?*
- **RQ3 (QoS Encoding and Robustness):** *(i) Do middleware Quality-of-Service contracts carry signal a purely structural score discards, (ii) how does that signal combine with architectural typing, (iii) do the framework’s simulation oracles agree with one another, and (iv) are the reported orderings robust to the free parameters of the scorer and of the ground truth?*
- **RQ4 (Out-of-Generator Transfer):** *How well does the framework transfer zero-shot to architecture models of five open-source systems spanning autonomous driving (ROS 2), cloud-native microservices, smart-home IoT, and industrial edge computing?*
- **RQ5 (Analysis Cost):** *What does pre-deployment analysis cost at CI/CD time, which pipeline stage dominates that footprint, and how does it compare with running the simulation directly?*

1.5. Key Contributions

This paper makes three main contributions:

1. **Learned failure-impact ranking over QoS-annotated architecture graphs:** Graph learning engines that read SaG’s typed multigraph with 16-D QoS edge encodings (Section 4), together with a capacity-matched control showing that the QoS channel, not relation-specific weights, drives their accuracy (Section 7.2). They transfer zero-shot to independently authored system models ($\rho = 0.76\text{--}0.81$ against $0.51\text{--}0.53$). A hybrid variant that learns a correction to SaG’s closed-form score (SaG-Hybrid) significantly outperforms closed-form ranking out of distribution ($+0.103$, 11/12 folds, Holm $p = 0.0068$; Section 7.5).
2. **A QoS-aware typed architecture model:** A multigraph representation that derives logical dependencies from physical pub-sub linkages, weights them by declared QoS contracts, and distinguishes sequential cascades from simultaneous multi-consumer failures (Section 3). Ranking on this projection improves over unweighted centrality on every held-out architecture ($+0.204$).
3. **A reproducible benchmark and cost profile:** Seventeen architectures totaling 2,812 components: twelve synthetic topologies that regenerate byte-identically from committed configurations, and five independently authored models of open-source systems (Sections 6–7). All reported table values are mechanically reconciled against the released artifacts. The pipeline’s cost profile shows that inference is negligible (56 ms, 0.02% of runtime) and that one structural metric dominates feature extraction (Section 7.6).

Relationship to the authors’ prior work. A previous conference paper [28] introduced the preliminary multigraph formulation and deterministic quality model on synthetic topologies. This JSS manuscript substantially extends that work through incorporating the complete predictive HGT pathway with 16-dimensional QoS edge encoding (Section 4); inductive LOSO cross-validation (Section 7.2); zero-shot evaluation on five architecture models of open-source systems (Section 7.4); empirical cost and sustainability characterization (Section 7.6); multi-oracle convergent validity and graph-view separation (Sections 4.3–4.4); and system-wide sensitivity analyses (Section 7.3). We retained formalisms from the conference paper only in restructured portions of Sections 3 and 5. On the larger corpus and oracles used here, the deterministic quality model of [28] is best used for attribution rather than ranking (Section 5).

1.6. Paper Organization

The rest of this paper is organized as follows: Section 2 reviews related work. Section 3 explains the SaG multigraph model and dependency projections. Section 4 describes the Heterogeneous Graph Transformer and simulation oracles. Section 5 presents the ISO/IEC-based explanation layer. Section 6 covers the experimental protocol, and Section 7 reports results for RQ1–RQ5. Section 8 discusses practical consequences, sustainability, threats to validity, and limitations. Section 9 concludes.

2. Related Work

This study brings together four main research areas: dependability, performance, and sustainability in distributed software systems; static code and system analysis; software quality analysis and multi-criteria evaluation; and graph representation learning with explainable AI. Together, these fields provide the basis for predicting cascading vulnerabilities from Architecture-as-Code descriptors before setting up any infrastructure.

2.1. Dependability, Performance, and Sustainability in Distributed Software Systems

Publish-subscribe (pub-sub) and asynchronous event-driven models separate communicating entities by space, time, and synchronization, enabling scalable, high-throughput systems [7]. Modern middleware standards like ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4] manage these exchanges using detailed Quality-of-Service (QoS) policies. In cloud-native microservice meshes and distributed AI or LLM serving systems, asynchronous message passing and queueing are the main ways components communicate, affecting latency, throughput, and hardware use.

Previous research on dependability and performance has focused on runtime methods such as dynamic consensus, broker clustering, backpressure throttling, autoscaling, and automated failover. It has also explored chaos engineering and runtime fault injection [23], which intentionally disrupt live clusters to study degradation and recovery. While runtime testing is realistic, it requires a working cluster, can disrupt services, and uses significant computing resources. These factors make it less practical during early design or commit-level CI/CD, and it is one of the development-time computations examined in green software engineering [29, 30, 31, 32, 33]. By contrast, this study examines dependability using Architecture-as-Code descriptors before setting up any infrastructure.

A long history of research has predicted dependability from architectural descriptions. For example, Cheung’s absorbing-Markov-chain model [34] calculates system reliability using component reliabilities and control flow graphs. Goseva-Popstojanova and Trivedi [35] organized later state-based, path-based, and additive models, as reviewed by Immonen and Niemelä [36]. Model-driven frameworks like the Palladio Component Model [37] and layered queueing networks [38] use component specifications to predict performance and reliability. Annotation-based methods, such as the AADL Error Model Annex [39], create fault trees from declared error states. However, these approaches cover broader topics than this study and need operational profiles and failure rates that are not available at commit time. Instead, SaG focuses on a specific question using only manifests: which components’ failures spread the furthest in the declared topology?

Many studies find faults in microservices by using operational telemetry. For example, Seer [40] and Sage [41] predict QoS problems from hardware counters and traces. MicroRCA [42] and TraceRCA [43] find root causes using service-dependency graphs. DeepTraLog [44], Eadro [45], and MicroCause [46] use graph neural networks on traces, logs, and metrics, as Zhang et al. [47] review. Zhou et al. [48] found that cascading outages in synchronous microservices often stem from thread-pool exhaustion, RPC timeouts, and repeated retry storms along call trees. In pub-sub systems, failures spread through broker queue saturation and message starvation. Because these methods require a running cluster to produce runtime telemetry, SaG works before deployment by analyzing static manifests before any code runs.

The question SaG asks — which components’ failure reaches furthest — is also the question of software change impact analysis [49], which traces how a modification propagates through dependency structure, and of dependency management in microservice fleets [50], where cyclic and undeclared dependencies are a known outage source. Ranking components by predicted impact, rather than classifying them, relates to learning-to-rank formulations of defect prediction [51], which optimize the ranking measure directly as our listwise loss does (Section 4.2).

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube [19]) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [20], class cohesion, module coupling (e.g., Lack of Cohesion in Methods [LCOM], Coupling Between Objects [CBO]) [21, 22], and code duplication to flag internal code smells and defect-prone modules [52, 53, 54, 55]. However, SCA cannot

observe runtime communication topology: it does not capture inter-service messaging channels, message broker queue saturation, or cross-host failure propagation.

Recovering system-level structure through static analysis is an active research area. Bushong et al. [56] create communication diagrams from static code analysis, and a recent review looks at nine architecture recovery tools [57]. In robotics and publish-subscribe systems, tools like HAROS [58] use static analysis to find structural problems before launching the system. Together, this research helps build accurate system models for better understanding and drift detection. Unlike these, SaG uses the declared topology to predict how failures might spread.

To close the "Architecture–Code Gap," Static System Analysis (SSA) expands static analysis from individual service code to the whole system architecture. By modeling distributed applications, message topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA spreads code-level quality metrics across architectural dependencies. As a result, engineering teams can find structural anti-patterns [59, 60] and architectural technical debt [61] early in the CI/CD process [62, 63], before flawed topologies reach production.

2.3. Software Quality Models and Multi-Criteria Evaluation

The ISO/IEC 25010:2023 product quality model [13] and the ISO/IEC 25019:2023 Quality-in-Use model [14] define software product quality. From the characteristics listed under Reliability, Maintainability, and Efficiency of Performance in ISO/IEC 25010:2023, SaG focuses on those derived from deployment topology: Availability, Fault Tolerance, Modularity, Modifiability, and Analyzability (Section 5.1). Accordingly, topological analysis does not cover other qualities such as Faultlessness, Recoverability, Reusability, and Testability.

Software engineering measures both internal quality, which refers to built-in structural features checked on static artifacts, and external quality, which covers runtime dependability and behavior during system operation [64, 65]. In distributed systems, technical debt such as over-centralized message topics or unreplicated brokers can lower internal quality and lead to serious external problems, including performance bottlenecks, queue congestion, and outages.

Combining different structural metrics into a single, auditable quality score is a classic Multi-Criteria Decision Making (MCDM) problem. The Analytic Hierarchy Process (AHP) [66] uses structured pairwise comparisons and a Consistency Ratio to ensure consistent judgments ($CR \leq 0.10$). However, while this ratio finds inconsistencies, it cannot spot matrices filled with preset answers, which this study addresses for the weights (Section 5.2). Therefore, AHP provides an audited, explainable Reliability–Maintainability (RM) quality baseline alongside learned graph models.

2.4. Graph Representation Learning and Explainable AI

Network science uses centrality indices such as degree, closeness, betweenness centrality [24, 26], articulation points, and PageRank [25, 27] to identify important nodes. Key studies on network robustness [10], cascading overloads [8], and interdependent networks [9] model how disruptions spread in connected systems. Percolation models are useful for comparing network breakdowns, but their thresholds assume homogeneous, undirected networks. Because these calculations do not apply to directed multigraphs with different entity types, channels, and QoS contracts, this study uses centrality-based baselines (Section 6.2). It suggests multi-type percolation fragmentation as a promising future approach.

Standard network measures have two main problems when applied to software architectures. First, Dimensional Collapse happens when a single centrality score cannot show why a component is critical, such as telling apart a single point of failure, a cascade hub, or an overused library. Second, Semantic Collapse occurs when unweighted metrics treat all nodes and edges the same, mixing up very different architectural elements like message topics, shared libraries, and physical hosts. Together, these problems motivate the core claim that richer architectural analysis is needed.

To overcome the limits of hand-crafted metrics, recent studies use machine learning for network vulnerability analysis and ranking node importance. Examples include FINDER [67], which applies deep reinforcement learning to find key nodes that break network connectivity; DrBC [68], which uses Graph

Neural Networks to estimate betweenness centrality in large networks; and PowerGraph [69], which models power grid risks. However, these models assume homogeneous, undirected, unweighted graphs. They cannot be used directly for distributed software systems because Architecture-as-Code manifests are heterogeneous: software topologies are directed multigraphs with several entity types, multiple relation types, and complex QoS contracts. Flattening these architectures into single-layer graphs for DrBC or FINDER causes severe Semantic Collapse and removes important edge contracts. As a result, existing homogeneous models (GCN [70], GraphSAGE [71], GAT [72]) average signals across all connection types, which discards relation identity unless it is supplied as a feature. Heterogeneous Graph Neural Networks (RGCN [73], HAN [74], HGT [75], MAGNN [76]) solve this by using relation-specific transformations. This study uses the Heterogeneous Graph Transformer (HGT) [75] with edge encodings to keep relational semantics when predicting cascade blast radii, and also tests matched homogeneous models (GAT-N, GAT-N-QoS) and closed-form baselines (Topo-QoS) to compare learning approaches (Section 6.2). Graph learning has also been used for microservice topologies; for example, Khodabandeh et al. [77] predict future service interactions using graph attention on segmented call graphs. However, that method predicts edge existence based on past interactions. In contrast, this study uses a declared topology as input and predicts the blast radius after removing a node.

A key challenge in using modern AI for software engineering is the black-box problem: deep neural models produce risk scores or embeddings without explaining the underlying causes. In real-world settings, these unclear risk rankings make it hard for developers and site reliability engineers (SREs) to decide whether to replicate hosts, set up circuit breakers, or refactor shared libraries.

Current graph neural network (GNN) explanation methods, such as GNNExplainer [78] and PGExplainer [79], identify important subgraphs by masking edges or using parameterized learning. While useful, these methods explain models using internal features instead of standard software engineering terms. SaG addresses this by using a dual-pathway design. Per-relation attention in the HGT pathway is a candidate diagnostic, although on our data its spread across relation types is narrow and largely driven by destination in-degree (Section 7.3.3). Meanwhile, the explanation layer links fragility to ISO/IEC quality sub-characteristics (Section 5), naming a remediation class for each flagged component.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 presents the end-to-end architecture of the SaG framework. The shared front end processes Architecture-as-Code manifests, constructs a typed multigraph, projects implicit runtime interactions throughout a QoS-weighted logical dependency layer, and extracts typed node properties. These features feed two independent pathways with no shared parameters. The predictive pathway (Section 4) forecasts cascading failure blast radii using a Heterogeneous Graph Transformer. The explanation layer (Section 5) decomposes fragility into Reliability and Maintainability quality profiles.

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature encodings (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system can be explicitly described as a typed, weighted, and directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- V is the set of system entities, divided into five categories: $\mathcal{T}_V = \{\text{app, broker, topic, host, lib}\}$, so $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$. To avoid confusion with physical machines, V_{host} refers to *Execution Hosts*, which can be physical or virtual nodes.
- E is the set of directed edges connecting entities.
- $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ are typing functions assigning entity and relationship categories.

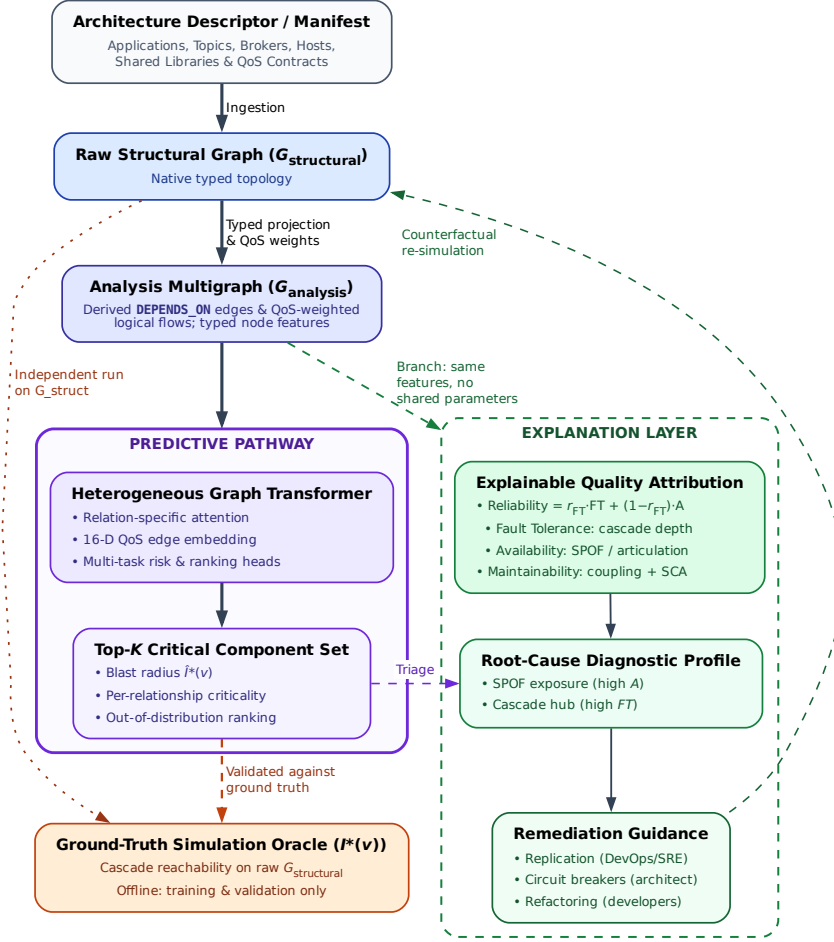


Figure 1: End-to-end architecture of the SaG framework. The predictive pathway runs left to right: manifest ingestion, typed multigraph, QoS-weighted `DEPENDS_ON` projection, typed node properties, heterogeneous graph learning, ranked critical set. The dashed edge marks the ground-truth simulation oracle, which operates only on $G_{\text{structural}}$, trains the predictor offline and takes no part in inference. The explanation layer re-enters from the analysis multigraph and shares no parameters with the predictor, reaching flagged components through triage rather than data flow.

- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ are weighting functions that show how critical an entity is and how strong a connection is. For applications and shared libraries, $w_V(v)$ is set using static code metrics: $w_V(v) = 1 - \text{CQP}(v)$ (Section 3.4). When static code metrics are unavailable, or for infrastructure entities, set the weight to 1.0 ($w_V(\text{host}) = 1.0$).

Tables 1 and 2 summarize the five entity types and six structural edge types in the SaG model, with representative distributed-system implementations.

Table 1: Entity types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Execution Host (V_{host})	Physical host or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime

Application and Library entities use static code metrics from Static Code Analysis (SCA) tools, including lines of code, cyclomatic complexity, coupling between objects, and method cohesion. Together, these metrics show how code-level fragility connects to topological analysis.

Table 2: Structural edge types in the SaG model.

Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO	App/Library $\rightarrow$ Topic	Component publishes messages to topic
SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component consumes messages from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on physical/virtual host
CONNECTS_TO	Host $\rightarrow$ Host	Physical network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library dependency

Table 3: Notation used throughout. Entity and edge types: Tables 1–2; simulation oracles: Table 5.

$G_{\text{structural}}$	Raw multigraph; oracles only	$Q(v)$	RM composite quality score
G_{analysis}	DEPENDS_ON projection; predictor input	ρ	Spearman ρ , full population
V_{app}	Application nodes; the scored population	$\rho_{>0}$	Spearman ρ , active stratum ($I^* > 0$)
$w(t), w(e)$	QoS topic weight, edge weight	Overlap@ K	Top- K set overlap, $K = 0.20 V_{\text{app}} $
$I^*(v)$	Primary cascade-reachability oracle	$I_{\text{comp}}, I_{\text{dyn}}, I_M$	Further oracles (Table 5)

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, a communication link’s strength depends on Quality-of-Service (QoS) contracts. For example, a RELIABLE topic with TRANSIENT_LOCAL durability creates a stronger connection between services than a BEST_EFFORT telemetry stream. This difference underpins the following weighting rules.

Each topic t carries an intrinsic criticality weight $w(t) \in (0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w(t) = \alpha_{\text{top}} \cdot \text{QoS}(t) + \beta_{\text{top}} \cdot \text{SizeNorm}(t) + \gamma_{\text{top}} \cdot \text{FreqNorm}(t), \quad (\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}}) = (0.75, 0.15, 0.10) \quad (2)$$

where $(\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}})$ is a convex combination satisfying $\alpha_{\text{top}} + \beta_{\text{top}} + \gamma_{\text{top}} = 1.0$. The QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24, 0.62, 0.14) \quad (3)$$

Here, $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport-priority scores mapped from manifest policies: $q_{\text{rel}} \in \{0.0, 1.0\}$ (best-effort vs. reliable), $q_{\text{dur}} \in \{0.0, 0.5, 1.0\}$ (volatile, transient-local, persistent), and $q_{\text{prio}} \in \{0.0, 0.5, 1.0\}$ (low, medium, high). Durability carries the highest weight because it determines state maintenance and historical message replay across node restarts. We distinguish this structural coupling weight $w(t)$ from the instantaneous cascade reachability oracle $I^*(v)$ (Section 4.3): while $I^*(v)$ scales immediate reachability loss via reliability and priority tiers, durability governs whether failure states persist and propagate across lifecycle boundaries. The sub-weights derive from a Saaty pairwise matrix with consistency ratio $CR = 0.016 \leq 0.10$. We note that CR is uninformative for a matrix back-filled from a chosen priority vector, so we state which case this is: the Topic QoS matrix is one of the two in the framework that carry genuine second-eigenvalue spread. Its CR therefore reports consistency rather than construction (Supplementary Table S4). The modulators $\text{SizeNorm}(t)$ and $\text{FreqNorm}(t)$ are logarithmically compressed and clamped to $[0, 1]$.

$$\text{SizeNorm}(t) = \min \left(1.0, \frac{\log_2(1 + B(t))}{20} \right), \quad \text{FreqNorm}(t) = \min \left(1.0, \frac{\log_{10}(1 + F(t))}{3} \right) \quad (4)$$

Here, $B(t)$ is the message payload size in bytes. The design envelope is 1 MiB, which is the practical DDS sample limit before RTPS fragmentation becomes a problem. $F(t)$ is the normal publication frequency in Hertz. The final weight $w(t)$ is limited to $[0.01, 1]$ so that best-effort edges remain visible during graph traversals. Each structural communication edge connected to t (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES) uses $w_E(e) = w(t)$ and the topic’s QoS vector. The full $(\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}})$ simplex keeps the orderings ($\rho \geq 0.919$), with small ranking changes of 0.031 (Topo-QoS) and 0.007 (RM). Morris screening shows that none of the three parameters are critical ($\mu^* \leq 0.025$), so the split is a documented choice, not a sensitive parameter.

3.2.1. Logical Dependency Projection (*DEPENDS_ON*)

Structural edges represent explicit deployment connections but do not capture implicit runtime dependencies. For example, a subscriber depends on a publisher, yet no direct edge connects them in publish-subscribe architectures. To tackle this limitation, a single unified semantic relation, *DEPENDS_ON*, is derived and directed from *dependent* to *dependency* ("if target fails, source is impacted"), following the six projection rules detailed in Table 4. The resulting weight $w \in (0, 1]$ quantifies the magnitude of operational coupling and indicates the conditional likelihood that a disruption in the dependency propagates to the dependent.

Table 4: The six *DEPENDS_ON* logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive USES)	$1 - \prod_{t \in T} (1 - w(t))$
2	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_{\text{DEPENDS_ON}}(u \rightarrow v)$
4	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_{\text{DEPENDS_ON}}(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6	broker_to_broker	Broker $\leftrightarrow$ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{host})$

Rules 1 and 2 combine the set of topics T connecting a pair of components using a probabilistic union instead of a maximum [80, 81, 82]. This method confirms that more parallel failure paths always increase coupling, while keeping w within $(0, 1]$. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [83] to combine the weights of the Application and the shared Library, balancing their importance. Rules 3 and 4 extend dependencies across hosts by using the highest coupling weight.

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A central principle of the SaG model is the distinction between two degradation modes. In a **Sequential Cascade (Rule 1)**, a failed publisher starves downstream subscribers sequentially through message queues and topic buffers. In a **Simultaneous Blast (Rule 5)**, a crashed library or execution host causes all consuming applications and colocated brokers to fail instantaneously in a single shared-fate event. Preserving entity types and relation-specific projection rules lets the SaG model represent both mechanisms, whereas untyped homogeneous graphs collapse them into indistinguishable edges.

Rule 6 is the only symmetric rule: two brokers colocated on the same host share that host's physical failure domain ($w = w_V(\text{host})$). Colocated brokers compete for resources, and host crashes halt them simultaneously. Because simulation oracles operate strictly on $G_{\text{structural}}$ (Section 4.4), Rule 6 has zero influence on ground-truth failure labels.

3.3. Dual Graph Views and Architectural Layers

The SaG framework consists of two distinct representations. The **Structural Graph** ($G_{\text{structural}}$) is the raw deployment graph containing physical and structural relations (*PUBLISHES_TO*, *ROUTES*, *RUNS_ON*, *USES*), preserving the untransformed deployment topology. The **Analysis Graph** (G_{analysis}) is the projected graph containing derived *DEPENDS_ON* edges annotated with QoS weights and ingested SCA metrics. We compute all GNN embeddings and analytical metrics on the analysis graph (Supplementary Figure S3).

G_{analysis} is further organized into four analytical layers (Application, Middleware, Infrastructure, and Global System), so that criticality can be evaluated per subsystem.

3.4. Typed Node Feature Encoding

Both the predictive pathway (Section 4) and the explanation layer (Section 5) read the same typed node properties from G_{analysis} : the predictor projects them per entity type before message passing, the explanation layer aggregates them into a quality profile. All five entity types share a common, deterministic 18-dimensional base block (indices 0–17) of topological metrics: (0) PageRank (*PR*), (1) Reverse PageRank (*RPR*), (2) Betweenness Centrality (*BT*), (3) Closeness Centrality (*CL*), (4) Eigenvector Centrality (*EV*), (5) In-Degree Centrality (*DG_{in}*), (6) Out-Degree Centrality (*DG_{out}*), (7) Local Clustering Coefficient (*CC*), (8) Undirected Articulation Score (*AP*), (9) Bridge Ratio (*BR*), (10) Node QoS Weight (w), (11) QoS-Weighted In-Degree (w_{in}), (12) QoS-Weighted Out-Degree (w_{out}), (13) Multi-Path Coupling Index (*MPCTI*), (14) Path Complexity (*PC*), (15) Fan-Out Criticality (*FOC*), (16) Directed Articulation Point (AP_c^{dir}), and

(17) Connectivity Degradation Index (*CDI*). The full mathematical schema, definitions, and normalizations for each of these eighteen metrics are detailed in Supplementary Table S11 (Supplementary Section S11). *CDI* is by far the most expensive of these and dominates the analysis cost of Section 7.6; because it is a predictor input and not only a term of the Availability score, it cannot be gated away without changing both pathways. PageRank, Reverse PageRank, betweenness and eigenvector centrality are computed on the QoS-weighted projection, so these four features carry QoS information into every predictor, including the arms reported as “QoS-off”, which lack only the explicit QoS edge channel and QoS node columns (10–12). Every metric in the block is normalized to $[0, 1]$ within its graph, which stops raw graph size from driving the per-type projections under cross-scenario transfer. Type-specific blocks extend the vector to 19–25 dimensions, adding source-code metrics and the Code Quality Penalty for Applications, reverse-USES blast-radius drivers for Libraries, queue capacity for Brokers, publisher/subscriber counts and ordinal QoS criticality for Topics, and CPU and memory allocation for Execution Hosts.

The shared block provides the GNN with global structural and positional context—analogous to positional and structural encodings in Graph Transformers—enabling relational message passing to modulate multi-hop representations based on global network role rather than local immediate adjacency alone. Crucially, because betweenness, closeness, reverse PageRank, and articulation scores are topological summaries computed before model evaluation, a learned model is not the only way to derive a criticality score from structure. The closed-form baselines are therefore competitive comparators rather than strawman alternatives, while making deterministic feature extraction the dominant computational bottleneck ($O(|V|^2 + |V||E|)$) analyzed in Section 7.6.

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in distributed software systems indicates non-linear, multi-hop, and relation-dependent characteristics. Outages propagate through architectural relations and dependencies beyond immediate neighbors. SaG therefore pairs its closed-form engine with a learned graph model that can combine structural cues across relation types; Section 7 evaluates both.

This section presents the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input–label independence guarantee designed to prevent data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Execution Hosts) and diverse interaction semantics (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES, DEPENDS_ON). We therefore employ a three-layer **Heterogeneous Graph Transformer (HGT)** architecture [75], implemented within PyTorch Geometric [84], with hidden dimension $D = 64$ and $H = 4$ attention heads. This architecture guarantees that typed relations, rather than simple adjacency, govern failure-impact forecasting.

4.1.1. Continuous-Categorical Edge Feature Encoding (16- D)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$. Index 0 is the scalar coupling weight $w_E(e) \in (0, 1]$ of Section 3.2; index 1 is the normalized count of simple paths through e ; indices 2–8 one-hot encode the seven structural and derived relations; and indices 9–15 carry middleware QoS parameters on PUBLISHES_TO and SUBSCRIBES_TO edges, zeroed elsewhere. Six QoS dimensions are active in our corpus — reliability, durability, message priority, a heterogeneity flag raised when an edge’s QoS triple departs from its scenario’s modal profile, and the deadline pair (an active flag and $\log_{10}(1 + \text{deadline_ns}/10^6)$, populated on 463 of 615 topics 75%). The seventh, $\log_{10}(1 + \text{max_blocking_ms})$, is a schema provision for hard real-time DDS and ROS 2 profiles and is zero throughout.

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}}e_{uv}$. Before relational attention computation, the current projection vector is incorporated directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$, SaG follows the Heterogeneous Graph Transformer formulation of Hu et al. [75] implemented via PyTorch Geometric’s HGTConv [84]. Entity-specific projections $W_{\tau(v)}$ first map raw features $x_v \in \mathbb{R}^{19-25}$ into the shared D -dimensional hidden space: $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. Relational mutual attention across H heads incorporates type-parameterized Key ($K(u) = h_u^{(l-1)}W_K^{\tau(u)}$), Query ($Q(v) = \tilde{h}_v^{(l-1)}W_Q^{\tau(v)}$), and Value ($V(u) = h_u^{(l-1)}W_V^{\tau(u)}$) projections along with the edge representation $\tilde{h}_v = h_v + e'_{uv}$. Crucially, attention scores scale by a learned per-meta-relation prior $\mu_{\langle\tau(u), \phi(e), \tau(v)\rangle}$ (`p_rel` in PyG), which lets the model weight an entire relation triple up or down independently of individual node embeddings; this parameter directly captures the relational typing effect evaluated in Section 7.2. Message passing operates bidirectionally across both forward and transposed relation views (G_{analysis} and $G_{\text{analysis}}^\top$) to capture downstream starvation and upstream backpressure simultaneously, followed by residual aggregation, dropout ($p = 0.10$), and layer normalization across layers $l \in \{1, \dots, L\}$.

Training Protocol and Optimization Hyperparameters. Models are optimized end-to-end with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4} , post-attention dropout 0.10) under cosine decay with warm restarts ($T_0 = 75$, $T_{\text{mult}} = 2$, $\eta_{\text{min}} = 3 \times 10^{-6}$), for up to 300 epochs with early stopping at patience 30 on validation loss over labeled nodes. Inductive subgraphs are packed per scenario without mini-batch subsampling, and validation masks isolate held-out nodes. Five seeds $\{42, 123, 456, 789, 2024\}$ are evaluated throughout, redrawing partitions and initializations for each. The architectural hyperparameters ($D = 64$, $H = 4$, dropout, learning rate, schedule) follow conventional HGT values [75]; the loss coefficients of Equation (5) were set by informed judgment, there being no convention for a five-term multi-task loss. Neither was tuned against the in-distribution test split or the LOSO folds, and no search was run over them. This avoids selection leakage but does not place either configuration near its own optimum: the comparison is between untuned configurations, and we state it as such.

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, a composite head predicts cascade impact, $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{a}_1(v) \parallel \hat{a}_2(v))) \in [0, 1]$, where $\hat{a}_1(v) = \sigma(\text{MLP}_1(h_v))$ and $\hat{a}_2(v) = \sigma(\text{MLP}_2(h_v))$ are two auxiliary heads. The code names them after Reliability and Maintainability, but in the evaluated configuration neither is supervised on its namesake: $\hat{a}_1$ is supervised on I^* and $\hat{a}_2$ is unsupervised, so both act only as learned feature enrichment for the composite head. An optional relationship-criticality edge head is disabled in every harness and is neither trained nor scored.

4.2.1. Optimization Objective and Dimension Masking

Under our headline experimental protocol, the active optimization objective directly balances cascade impact regression accuracy, auxiliary representation learning, global listwise ranking monotonicity, and fine-grained pairwise margin constraints:

$$\mathcal{L}_{\text{active}} = \text{MSE}(\hat{I}^*(v), I^*(v)) + 0.5 \cdot \text{MSE}(\hat{a}_1(v), I^*(v)) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} \quad (5)$$

where $I^*(v)$ is the simulated cascade impact defined by the primary oracle (Section 4.3), $\mathcal{L}_{\text{rank}}$ is the ListMLE listwise ranking loss [85] parameterized by temperature τ :

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\frac{\hat{s}_{\pi_i}}{\tau} - \log \sum_{j=i}^N \exp \left(\frac{\hat{s}_{\pi_j}}{\tau} \right) \right) \quad (6)$$

where $\pi = (\pi_1, \dots, \pi_N)$ denotes the permutation of nodes sorted in descending order of ground-truth impact $I^*(v)$, and $\hat{s}_v = \hat{I}^*(v)$. At the baseline default $\tau = 1.0$, the formulation reduces to standard ListMLE; the temperature parameter $\tau < 1.0$ is a configurable hyperparameter that sharpens probability distributions over narrow prediction margins. Pairwise ordering fidelity is guided by margin-ranking loss $\mathcal{L}_{\text{pairwise}} = \frac{1}{|P|} \sum_{(u,v) \in P} \max(0, \gamma - (\hat{s}_u - \hat{s}_v))$ with margin $\gamma = 0.05$ over pairs $P = \{(u, v) \mid I^*(u) - I^*(v) > \gamma\}$. Permutation-level ListMLE ($\mathcal{L}_{\text{rank}}$) provides the primary gradient force for global rank monotonicity (ρ), while pairwise margin ($\mathcal{L}_{\text{pairwise}}$) penalizes small-margin inversions among adjacent components.

The implementation also supports a general multi-task objective with a maintainability term and an optional consistency term tying the auxiliary heads to the explanation layer. Both are switched off throughout ($m_M = 0$, $\lambda_{\text{RM}} = 0$), so the two pathways share no parameters and Eq. (5) is the objective actually optimized. The general form is given in Supplementary Section S20 for completeness.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy before deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. Table 5 summarizes the four component-level oracles and their distinct roles across the evaluation program.

Table 5: Simulation oracles, operational constructs, and evaluation roles.

Oracle	Physical Mechanism	Nature	Role in Evaluation
$I^*(v)$	BFS cascade reachability + QoS ladder	Seeded tie-breaking	Primary ranking target (RQ1, RQ2, RQ4)
$I_{\text{comp}}(v)$	Severity mixture: reachability + fragmentation	Deterministic	Explanation layer / Validate gate
$I_{\text{dyn}}(v)$	Discrete-event SimPy message queuing	Stochastic	Convergent-validity probe (RQ3)
$I_M(v)$	Reverse DEPENDS_ON traversal	Deterministic	Unsupervised maintainability reference

- Cascade Reachability Oracle ($I^*(v)$)**, evaluated via discrete-event cascade fault injection: crashes component v , propagates outages across dependent topics, brokers, and links by breadth-first traversal, and returns the mean fractional feed loss over the subscriber population, each subscriber contributing the unweighted mean loss of the topics it subscribes to. A topic’s feed loss is the fraction of its publishers that have failed (for a topic with no publisher, the fraction of its failed routers), scaled by a QoS ladder ($\times 1.2$ RELIABLE, $\times 1.15$ high/urgent priority, $\times 1.05$ medium) and clamped to $[0, 1]$. The ladder encodes a severity judgment rather than a physical mechanism: a subscriber that declared a reliable or high-priority contract depends on every sample, so losing that feed is scored as more harmful than losing a best-effort one. The factors are declared constants, not calibrated values; removing them leaves the Application ordering nearly unchanged (below). The denominator is the full subscriber set of the intact graph, so subscribers that themselves fail are retained in the average rather than excluded. The implementation admits a per-publisher rate weighting, but rates are declared per topic throughout our corpus, so it reduces exactly to this publisher fraction. Five seeds $\{42, 123, 456, 789, 2024\}$ drive a seeded shuffle that breaks ties in wave-propagation order, and $I^*(v)$ is the mean over them. The label is therefore reproducible given the seed set but not seed-invariant, which is what the label-noise ceiling of Section 7.1 measures. This is the **primary continuous target label** throughout.

How much QoS is in this label. The ladder reads reliability and transport priority only; durability never enters I^* , despite carrying the largest of the three elicited sub-weights (0.62; Section 3.2). That omission does not bound the label’s QoS content, because disabling QoS scaling entirely leaves the Application ordering nearly intact — mean Spearman $\rho = 0.965$ against the ladder across the twelve folds (range 0.891–0.999) — and substituting a durability-aware $w(t)$ scaling moves it less still ($\rho = 0.977$). The top- K set is the sensitive construct: ladder and topology-only labels agree at mean Jaccard 0.678, so QoS changes *which* components are named critical without changing their order. I^* is therefore a near-topological target carrying a QoS term at its threshold boundaries, which bounds what any QoS-encoding result can be crediting (Section 7.3.1).
- Multi-Metric Composite Oracle ($I_{\text{comp}}(v)$)**, evaluated via multi-metric failure simulation: a severity-weighted mixture of reachability loss, fragmentation, throughput loss and flow disruption, with AHP-derived coefficients (0.35, 0.25, 0.25, 0.15). Those coefficients come from a rank-one comparison

matrix, so they record their origin without independently justifying them. They are not swept in our sensitivity analysis — a gap worth naming because I_{comp} supplies the labels for the explanation layer’s evaluation. It is reserved for Validate-stage gates and prescriptive verification, never for forecasting ranking.

- **Dynamic Queue-Flow Oracle ($I_{\text{dyn}}(v)$)**, evaluated via discrete-event message-flow queue simulation (built on SimPy [86]): simulates emission rates, stochastic latencies, broker buffer saturation, and queue drops under fault injection, extracting the drop in delivered message rate to surviving consumers. The engine instruments Google SRE’s *Four Golden Signals* across pre- and post-fault windows — latency decomposed into queue wait and service time at p50/p95/p99, traffic in Hz and KB/s, errors spanning deadline violations and overflow discards, and time-weighted mean queue depth for saturation — as diagnostic telemetry. They are kept out of $I_{\text{dyn}}(v)$ rather than mixed into it, because crashing a high-rate publisher clears downstream subscriber queues (*contention relief*, $\rho = -0.499$ between delivery loss and tail latency delta). An additive composite would mathematically cancel delivery damage with latency reduction. $I_{\text{dyn}}(v)$ is therefore kept strictly 1-dimensional, serving as an independent convergent-validity probe (Section 7.3.2).
- **Change-Propagation Oracle ($I_M(v)$)**, evaluated via structural change-propagation analysis: a deterministic reverse-dependency traversal over the transpose of the six-rule DEPENDS_ON projection, blending change reach, weighted change impact, and normalized depth. It is a structural maintainability reference and is never used as a training label, which would make the supervision circular.

Topic Criticality Label Masking. The multi-metric failure simulator can incorporate declared topic criticality into its severity term. This feature is disabled, however, because topic criticality is a GNN input feature: using it would result in the predictor being measured against a transformation of its own input.

Primary Oracle Declaration and Role Assignment. The three reliability-facing oracles measure separate constructs, so we designate $I^*(v)$ **the primary continuous target** for all predictive ranking results (Tables 8–9, RQ1–RQ3): it is reproducible from a fixed seed set, as CI/CD gating requires, and its seed-to-seed variation is limited to tie-breaking, whereas I_{dyn} adds stochastic latencies, bursty arrivals and synthetic buffer limits. That choice has a cost, since it makes the target a topological functional of the predictors’ own input (Section 4.4). $I_{\text{comp}}(v)$ is reserved for Validate-stage gates and prescriptive verification, $I_{\text{dyn}}(v)$ is an independent convergent-validity probe (Section 7.3.2), and $I_M(v)$ a structural maintainability reference.

Cross-Oracle Convergent Validity. As detailed in Section 7.3.2, the three reliability oracles show substantial but sub-ceiling agreement on Applications ($\rho = 0.627$ for (I_{dyn}, I^*) against a 0.811–1.000 label noise floor), confirming distinct constructs. Consequently, results established against one oracle are never transferred to another; every evaluation explicitly references its underlying simulation oracle.

4.4. Input-Label Independence Guarantee

To prevent data leakage, SaG applies strict architectural separation. Feature Space is constructed exclusively from G_{analysis} using static structural topology, static code metrics, and declared QoS contracts. Label Space is evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (cascade reachability injection, composite failure simulation, and dynamic message-flow simulation). No simulation outputs, failure trace histories, or dynamic execution telemetry are exposed as input attributes to the GNN or the explanation layer.

Scope of the guarantee. The separation rules out circular feature construction: no predictor reads a transformation of the quantity it is scored against. Because G_{analysis} is itself a projection of $G_{\text{structural}}$, features and labels still derive from the same topology, and $I^*(v)$ is a reachability functional of it. The learning task is therefore to synthesize pre-computed structural cues across heterogeneous relations into a ranking that matches the simulator, which is also why SaG’s closed-form engine is a strong reference point (Section 7.1). All labels are simulator-derived; validation against field incidents is discussed in Section 8.3.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictor of Section 4 locates risk but says nothing about remedy. A component may be critical because it is an unreplicated single point of failure, an error-propagating cascade hub, or a high-coupling maintainability bottleneck, and each calls for a distinct intervention: replication, circuit breaker insertion, or architectural decoupling. This section formalizes the explanation layer that attributes those structural causes, decomposing criticality into a standards-grounded quality profile computed from the same typed node properties (Section 3.4) without sharing parameters with the predictor, and reaching flagged components through triage rather than data flow (Figure 1).

Role. The explanation layer is a diagnostic attribution instrument applied after ranking: the engines of Sections 4 and 6.2 decide *which* components are critical, and this layer explains *why*, in the vocabulary of ISO/IEC 25010/25019. It is not used as a ranker; its ranking correlation is reported for reference in Table 8, and its weight sensitivity in Supplementary Section S1.

5.1. Grounding in ISO/IEC Standards

In accordance with ISO/IEC 25010:2023 [13] and ISO/IEC 25019:2023 [14], SaG formalizes two primary criticality dimensions. Component Criticality (D_1) is service loss upon component failure; Relationship Criticality (D_2) is service decline upon channel severance.

Criticality is assessed across two orthogonal characteristics: **Reliability** (R) and **Maintainability** (M). Reliability is divided into **Fault Tolerance** (FT) and **Availability** (A). FT uses Reverse PageRank, in-degree, and cascade depth potential to inform redundancy and circuit breaker strategies. A uses directed articulation points, bridge ratios, and connectivity degradation to inform replication strategies. Maintainability (M) assesses structural coupling and code-level complexity, using betweenness, QoS-weighted fan-out, code quality penalties, and clustering to guide decoupling and refactoring. This partition maps each ISO/IEC sub-characteristic to its graph metrics and remediation roles. Safety and security considerations that require specialized hazard logs are excluded from purely structural topology analysis.

5.2. Composite Quality Score Formulation

All raw metrics are rank-normalized to the interval $[0, 1]$ within the graph. Quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [66]:

- **Fault Tolerance** ($FT(v)$): Evaluates error cascade potential on transpose graph $G_{\text{analysis}}^\top$: $FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v)$, where $\text{RPR}(v)$ is Reverse PageRank, $\text{Deg}_{\text{in}}(v) = d_{\text{in}}(v)/(|V| - 1)$ is normalized in-degree, and $\text{CDPot}_{\text{enh}}(v) = \text{depth}(v)/\max_{u \in V} \text{depth}(u)$ is normalized cascade depth potential on $G_{\text{analysis}}^\top$.
- **Availability** ($A(v)$): Identifies structural single points of failure across five terms: $A(v) = 0.25 \cdot \text{AP}_c^{\text{dir}}(v) + 0.20 \cdot \text{QSPOF}(v) + 0.20 \cdot \text{BR}(v) + 0.25 \cdot \text{CDI}(v) + 0.10 \cdot w(v)$, where $\text{AP}_c^{\text{dir}}(v)$ is Directed Articulation Point severity, $\text{QSPOF}(v)$ is QoS-weighted SPOF severity, $\text{BR}(v)$ is Bridge Ratio, $\text{CDI}(v)$ is Connectivity Degradation Index, and $w(v)$ is intrinsic QoS weight.
- **Reliability** ($R(v)$): Blends Fault Tolerance and Availability: $R(v) = r_{\text{FT}} \cdot FT(v) + (1 - r_{\text{FT}}) \cdot A(v)$ with $r_{\text{FT}} = 0.36$. Intra-dimension weights apply $\lambda = 0.70$ shrinkage blending with a uniform prior. Three of the framework’s five comparison matrices (Impact, Maintainability, Availability) are rank-one by construction, so their Consistency Ratios are uninformative; the remaining two (Topic QoS and Fault Tolerance) carry genuine second-eigenvalue spread and their CR figures do mean what CR normally means. Supplementary Section S4 separates them. A uniform prior ($\lambda = 0$) yields a higher ranking correlation (0.319 vs. 0.200; Supplementary Section S1) and is the recommended setting when $Q(v)$ is used to rank; the default $\lambda = 0.70$ is kept because the same constant parameterises the I_{comp} severity weights.
- **Maintainability** ($M(v)$): Blends structural coupling with static code analysis: $M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v))$, where $\text{BT}(v)$ is Betweenness Centrality, $w_{\text{out}}(v)$ is QoS-weighted efferent coupling, $\text{CQP}(v)$ is Code Quality Penalty, and $\text{CC}(v)$ is local Clustering Coefficient.

The baseline composite quality score integrates both dimensions as follows: $Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v)$. When evaluated under an ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is dynamically reweighted: $Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v)$. Components are partitioned into Tukey tiers: CRITICAL ($Q > Q_3 + 1.5 \cdot \text{IQR}$), HIGH, MEDIUM, and MINIMAL. Across the benchmark topologies, this conservative Tukey upper fence flags an empirical mean of 4.2% of components (range 1.8%–8.3%), deliberately isolating the extreme right tail of architectural risk to prioritize developer intervention. High Availability (A) combined with low Fault Tolerance (FT) indicates a single point of failure that necessitates replication. In contrast, high Fault Tolerance (FT) identifies an error-cascade hub that requires circuit breakers (Section 8.4). Supplementary Table S16 shows a representative Diagnostic Remediation Card, illustrating how the sub-characteristics translate topological metrics into refactoring actions.

5.3. Prescriptive Remediation and Counterfactual Verification

After attributing root causes, candidate repairs (broker replication, circuit breaker insertion, topic decoupling) are generated and counterfactually verified in memory. Mutations are accepted only if they reduce systemic impact beyond simulation seed noise ($\Delta \bar{I}_{\text{comp}} > \kappa \cdot \sigma_{\text{seed}}$, $\kappa \geq 1.0$) without introducing new articulation points. This counterfactual verification loop illustrates the architectural pattern linking diagnosis to remediation; formal developer studies and automated patch synthesis benchmarks are reserved for future work.

6. Experimental Setup

6.1. Datasets and System Corpus

The evaluation corpus comprises 2,812 components across seventeen system architectures, including twelve synthetic topologies that form the inductive cross-validation folds and five architecture models of open-source systems withheld from all training procedures, as detailed in Table 6. The synthetic scenarios span diverse operational domains (autonomous vehicles, financial trading, healthcare integration, industrial SCADA, smart-city IoT, telecom RAN, cloud microservices, and enterprise application integration via centralized broker hubs/ESB; detailed in Supplementary Table S12). All twelve come from one generator family, as do their static code metrics, so leave-one-scenario-out evaluation measures transfer across scenario configurations of that generator rather than across independently produced architectures.

Table 6: Overview of the evaluation corpus. The twelve synthetic topologies correspond to the inductive Leave-One-Scenario-Out folds described in Table 8, and the five open-source system models are excluded from all training folds and used exclusively for zero-shot transfer (Section 7.4). Per-scenario entity and edge counts are obtained from the committed topology files and verified through continuous integration.

Dataset	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (11)	2,387	1,295	588	60	194	250	10,657
Synthetic case study — ATM (1)	74	26	27	5	8	8	261
Synthetic subtotal (12 LOSO folds)	2,461	1,321	615	65	202	258	10,918
Open-source system models (5)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

The five open-source systems are hand-authored models. Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and two e-commerce and booking meshes modelled after Online Boutique and Train-Ticket were each written by one author as a typed multigraph (`saag/adapters/realworld_adapter.py`), from the systems’ public documentation. They are not mechanical extractions from manifests. Brokers, QoS profiles, code metrics and host specifications are partly assumed, and two models depart materially from their originals: the Online Boutique model is a 22-application publish–subscribe mesh with Kafka, RabbitMQ, Redis and NATS brokers, whereas the original is a set of about eleven gRPC services without a message broker, and the Train-Ticket model represents its service-discovery server as a broker. Neither model contains a synchronous call edge. We therefore name these systems “modelled after” their originals and read RQ4 as transfer to independently authored architecture models, not to deployed systems.

6.1.1. Reproducibility of the Corpus

The benchmark corpus is designed to be fully regenerable rather than statically archived. Each dataset is deterministically generated from its configuration file via `python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <path>`. A companion manifest records the random seed, entity counts, git commit hash, and a SHA-256 cryptographic digest for each emitted topology. Continuous integration regression tests verify that every committed dataset regenerates byte-identically from its configuration and that all disk digests match the manifest. This procedure makes sure that third parties can reproduce the exact graphs used in these experiments, rather than sampling from similar distributions.

6.2. Baselines and Evaluated Predictors

SaG provides two ranking engines built on its QoS-aware projection — a closed-form engine (Topo-QoS) and a learned heterogeneous engine (HGT-QoS) — plus the hybrid of Section 7.5. They are evaluated against standard practice: unweighted centrality (Topo) and untyped graph neural networks (GAT-N, GAT-N-QoS). Table 7 provides a structured taxonomy of the evaluated predictors, their underlying graph representations, edge feature encodings, and empirical roles. Predictor names indicate the model family and substrate: an -N infix denotes a model trained on the complete native multigraph, its absence denotes the obtained Application-Library flow projection, and a -QoS suffix denotes a configuration that consumes declared QoS contracts. SaG throughout denotes the overall framework, not an individual predictor.

Table 7: Taxonomy of evaluated predictors and reference baselines. The -N infix denotes execution on the complete native multigraph; -QoS indicates inclusion of middleware QoS attributes.

Predictor	Evaluation Substrate	Typing	Edge Features	Parameters	Trained?	Empirical Role
Topo	G_{analysis} (Flow Projection)	No	None	0	No	Standard centrality baseline
Topo-QoS	G_{analysis} (Flow Projection)	No	Scalar $w(e)$	0	No	SaG closed-form engine
GAT	G_{analysis} (Flow Projection)	Homogeneous	None	28,168	Yes	In-distribution counterpart of GAT-N (Supp. Table S15)
GAT-QoS	G_{analysis} (Flow Projection)	Homogeneous	Scalar $w(e)$	28,168	Yes	In-distribution counterpart of GAT-N-QoS (Supp. Table S15)
GAT-N	Native Multigraph	Homogeneous	None	28,168	Yes	Untyped, unweighted GNN floor
GAT-N-QoS	Native Multigraph	Homogeneous	Scalar $w(e)$	28,168	Yes	Isolates QoS channel without typing
GAT-N-C	Native Multigraph	Homogeneous	None	437,496	Yes	Capacity-matched untyped control
GAT-N-QoS16-C	Native Multigraph	Homogeneous	16-D QoS Vector	429,992	Yes	Capacity- and channel-matched control
HGT	Native Multigraph	Heterogeneous	Relation 1-hot; $w(e)=1$	434,620	Yes	Isolates relational typing without QoS
HGT-QoS	Native Multigraph	Heterogeneous	16-D QoS Vector	434,620	Yes	SaG learned engine
SaG-Hybrid	Native Multigraph	Heterogeneous	16-D QoS Vector	434,941	Yes	Learned correction to the closed-form engine
RM / $Q(v)$	G_{analysis} (Analysis Graph)	Per-type rules	Scalar $w(e)$	0	No	Diagnostic attribution reference

Table 7 summarizes the evaluated predictors, graph representations, and empirical roles across three families: heterogeneous graph learning (HGT-QoS with 16-D QoS edge vectors, and its ablation HGT), homogeneous graph learning (GAT-N-QoS with scalar $w(e)$, and unweighted GAT-N), and training-free structural baselines (Topo-QoS and unweighted Topo on the DEPENDS_ON flow projection). The -N infix denotes the native multigraph substrate; on flow projections, homogeneous variants are denoted GAT/GAT-QoS (Supplementary Table S15). The out-of-distribution evaluation (Table 8) additionally reports RM ($Q(v)$, Section 5) as a diagnostic reference baseline. Deterministic RM scoring also drives sensitivity sweeps in Section 7.3.

6.2.1. Evaluation Substrates and Parity Guarantee

Predictors in this study operate on matched substrates within their respective families:

Graph Learning Models (GAT-N-QoS, HGT-QoS): Under Leave-One-Scenario-Out (Table 8), both learned predictors ingest the complete native typed multigraph across all five entity types (recorded using the shared -N infix). In-distribution (Supplementary Table S15), GAT/GAT-QoS consume the Application-Library DEPENDS_ON projection, confounding typing with multi-entity visibility. GAT-N-QoS uses per-type projection with untyped GATConv across edges, whereas HGT-QoS uses relation-specific HGTCov weights. The edge channel also differs: GAT-N-QoS consumes a scalar $w(e)$, while HGT-QoS consumes all 16 dimensions. The parameter budget (434,620 vs. 28,168) and message directionality (103,725 reverse parameters) also differ. In QoS-off arms every edge weight is set to 1 and the QoS node columns are zeroed, but four node centralities are computed on the QoS-weighted projection (Section 3.4), so “QoS-off” means “no explicit QoS channel”, not “no QoS information”. Four control arms (GAT-N-C, GAT-N-QoS-C, GAT-N-QoS16-C, HGT-QoS-U) are registered in Amendment 2 of our analysis plan and implemented; two of them (GAT-N-C, GAT-N-QoS16-C) with HGT and HGT-QoS form a 2×2 matched in capacity and edge-channel width, run as one paired CPU

sweep (`make -f reproduce/Makefile rq2-matched`; Table 11). The capacity-only QoS control and the directionality control (HGT-QoS-U) have not been run.

Closed-form scores (Topo, Topo-QoS): Both are computed on the Application-Library `DEPENDS_ON` projection (Section 3.2), because on the raw multigraph messages route through topics and brokers and Application betweenness vanishes. They are specified as a path-traversal term plus a cut-vertex term:

$$\text{Topo}(v) = 0.6 \cdot \text{BT}(v) + 0.4 \cdot \text{AP}(v), \quad \text{Topo-QoS}(v) = 0.6 \cdot \text{BT}_w(v) + 0.4 \cdot \text{AP}(v) \quad (7)$$

where $\text{BT}(v)$ is normalized betweenness centrality on the projection, $\text{AP}(v) \in \{0, 1\}$ indicates whether v is an articulation point of the projection’s undirected form, and $\text{BT}_w(v)$ is betweenness computed over edge distances $d(e) = 1/(w(e) + \varepsilon)$ with $\varepsilon = 10^{-6}$, so that strongly coupled edges are short and attract shortest paths. The projection itself carries two of the six rules of Table 4: Rule 1 joins a subscriber to each publisher of a topic it consumes, at $w = 1 - \prod_{t \in T} (1 - w(t))$, and Rule 5 joins an application to each library it uses. On this projection the Rule 5 edge carries a neutral constant, the graph’s median topic weight, rather than Table 4’s harmonic coupling of code-quality weights, so that the closed-form baselines consume no code metrics.

In the evaluated implementation the articulation term reads zero for every node, because the cached structural metrics carry no articulation score. The scores reported throughout are therefore $0.6 \cdot \text{BT}$ and $0.6 \cdot \text{BT}_w$, which rank components exactly as betweenness and QoS-weighted betweenness do. Restoring the articulation term lowers both on this corpus (Topo-QoS $0.553 \rightarrow 0.533$, Topo $0.349 \rightarrow 0.329$; Supplementary Section S22), so the evaluated form is the stronger reference and is kept as registered. The whole Topo-QoS gain over Topo therefore comes from SaG’s QoS weighting of shortest paths. When no edge of a graph carries a non-unit weight, the two coincide. The 0.6/0.4 split is a declared convention and is not tuned.

Ground-Truth Independence Guarantee: Crucially, **no predictor in either group accesses $G_{\text{structural}}$** : that raw topology is strictly reserved for simulation oracles (Section 4.4), verified by `tests/test_independence_guarantee.py`. Regardless of substrate, all variants are scored on the same independently resolved Application node set (V_{app} , Section 6.3).

6.3. Evaluation Metrics and Protocols

Figure accessibility. All figures employ the Okabe-Ito palette with distinct markers and hatchings to ensure monochrome legibility.

Ranking Precision: Evaluated via Spearman ρ against the simulated impact $I^*(v)$ from the primary oracle (Section 4.3). **Critical-Set Identification:** Measured via $\text{Overlap}@K$, the fraction of the true top- K components that the predicted top- K recovers ($K = \text{round}(0.20 \cdot |V_{\text{app}}|)$). Because both sets have exactly K members, top- K precision, recall and F_1 all equal this overlap. Identification at operating points where precision and recall are free to differ — $F_1@7$ against the labels’ own critical set, threshold-free PR-AUC, and rank-weighted nDCG@10 — is reported separately in Supplementary Table S13, because those quantities answer a question $\text{Overlap}@K$ structurally cannot. **Statistical Significance:** Paired Wilcoxon signed-rank tests [87] ($p < 0.05$) and bootstrap 95% CIs ($B = 2,000$) over folds [88, 89]. In the 12-fold LOSO design, the smallest attainable two-sided p is 0.00049. Folds share ten of eleven training scenarios, so these tests are anti-conservative; a corrected resampled test [90] would widen them, and we read all p -values as nominal (Section 8.3).

One confirmatory family, and everything else exploratory. The confirmatory family is the registered one and contains two contrasts: HGT-QoS against Topo-QoS, and HGT against Topo-QoS, Holm-corrected across those two alone. Neither reaches $\alpha = 0.05$ (Section 7.1). Every other contrast in this paper — the remaining full-population comparisons against Topo-QoS, the three orthogonal quantities of the 2×2 (Table 10), and the four simple effects — was formulated after the results existed and is reported as exploratory. The 2×2 quantities are Holm-corrected within their own block, labelled post-hoc where it appears. A second registered family, fixed before its run (Amendment 5), contains the two SaG-Hybrid contrasts — against Topo-QoS and against HGT-QoS — Holm-corrected across those two (Section 7.5). The capacity- and channel-matched 2×2 of Amendment 2 is likewise Holm-corrected within its own block (Table 11). For orientation, the largest exploratory effect is Topo-QoS over Topo ($+0.204$, $12/12$, $p = 0.0005$).

Registered analysis plan. The primary out-of-distribution contrast (HGT-QoS vs. Topo-QoS under LOSO, 5 fixed seeds, fold as unit of analysis) was registered in the replication package, with its protocol, statistic, unit of analysis and reporting commitment fixed, before the revised harness produced any result. We call it registered rather than pre-registered: the plan is a file in our own repository with no third-party timestamp, and an earlier, withdrawn eight-fold estimate of the same contrast predates it. The registration shows that the analysis was not selected after seeing the twelve-fold result, not that the question was asked without any prior estimate.

6.3.1. Evaluation Population and Protocols

Each predictor within an evaluation table is scored on an identical node population, resolved strictly from scenario topology and ground truth, specifically the **Application** set (V_{app}) unless otherwise noted. Pooling node types conflates distinct base rates and can trigger Simpson’s paradox (Section 7.3).

In-Distribution Evaluation: Stratified 60% train / 20% val / 20% test node splits over five seeds {42, 123, 456, 789, 2024}, redrawing partitions and initializations. **Inductive LOSO Cross-Validation:** Models are trained on eleven scenarios and test zero-shot on the held-out twelfth across all 12 folds under equal 3-layer depth and inner-split early stopping (Section 8.4). **Out-of-Generator Transfer:** Synthetic-trained models are evaluated zero-shot, without fine-tuning, on the five open-source system models.

7. Results and Empirical Analysis

Empirical results for RQ1–RQ5 are presented across the twelve-fold inductive benchmark and five architecture models of open-source systems. The evaluated populations are strictly stratified on the Application service set (V_{app}) under the input–label independence guarantee (Section 4.4).

7.1. RQ1: Graph Learning vs. Structural Baselines

RQ1 Summary (Predictive Efficacy):. Both SaG engines clearly outperform standard practice out of distribution. SaG’s QoS-weighted closed-form engine (Topo-QoS, $\rho = 0.553$) improves on unweighted centrality on all twelve held-out architectures (+0.204, $p = 0.0005$), and the learned heterogeneous engine (HGT-QoS) attains the highest mean correlation in Table 8 ($\rho = 0.638$). On their own, the two SaG engines are statistically on par (+0.085, $p = 0.151$, Holm 0.303); combined in SaG-Hybrid they significantly outperform the closed-form engine (+0.103, 11/12 folds, Holm $p = 0.0068$; Section 7.5).

What the in-distribution results can and cannot support. Per-scenario in-distribution figures are reported in Supplementary Table S15 rather than here, because no comparison can be drawn down their columns: GAT/GAT-QoS consume the Application–Library projection while HGT/HGT-QoS consume the native multigraph (Section 6.2.1), so a difference between the families confounds message passing with multi-entity visibility. Within a predictor the cells are still informative. In Healthcare, Topo-QoS achieves $\rho = 0.399$ but fails at critical triage ($\text{Overlap}@K = 0.000$); in the synthetic Microservices fold, HGT degrades ($\rho = 0.141$, $\text{Overlap}@K = 0.300$) where HGT-QoS does not ($\rho = 0.664$, $\text{Overlap}@K = 0.600$).

7.1.1. Out-of-Distribution (LOSO) Generalization

Inductive Leave-One-Scenario-Out cross-validation asks each model to predict cascading criticality on an entirely unseen topology, and the twelve folds are this paper’s primary anchor for generalization across architectural archetypes. Per-fold breakdowns are in the Supplementary Material (Table S12, Section S10); the main text reports the cross-fold summary (Table 8) and the active-stratum contrast (Table 9):

Each of the twelve folds holds out one scenario for zero-shot testing and trains on the remaining eleven, with all variants scored on the same Application node set (26 to 300 nodes, giving K between 5 and 60) and paired Wilcoxon tests over folds. For $\text{Overlap}@K$, HGT-QoS achieved 0.424 compared to Topo-QoS’s 0.388 ($\Delta = +0.037$, prevailing in 7 of 12 folds, $W = 29.0$, $p = 0.470$). The untyped GAT-N-QoS attained 0.431 ($\Delta = -0.006$, prevailing in 5 of 12 folds, 1 tie, $W = 27.5$, $p = 0.653$), indicating that critical-set identification does not statistically distinguish the typed model from either untyped learning or the QoS-weighted baseline.

Table 8: Inductive LOSO evaluation, Application population, twelve folds. Learned variants share training set, the native multigraph substrate (hence the -N infix; Section 6.2), depth, and selection rule (Section 6.3), differing in typing and edge channel; parameter budget, message-passing directionality and edge-channel width remain unmatched, and neither this table nor Table 10 controls for them (Section 8.3). Fold score = mean over five seeds; **Fold** σ = spread of the twelve fold means; **Seed** σ = median within-fold spread (zero by construction for deterministic baselines); CI = bootstrap over folds ($B = 2,000$). $\Delta\rho$ is paired by fold against **Topo-QoS**, the registered comparator (Section 6.3), with its own bootstrap interval; an interval spanning zero means the contrast is not resolved at twelve folds, which is the case for every learned variant.

Predictor / Reference	Mean LOSO ρ	95% CI	$\Delta\rho$ vs Topo-QoS	Fold σ	Seed σ	Overlap@K	Requires Training
<i>Training-free structural baselines</i>							
Topo	0.349	[0.254, 0.452]	-0.204 [-0.286, -0.122]	0.173	—	0.366	No
Topo-QoS	0.553	[0.443, 0.657]	— (reference)	0.192	—	0.388	No
<i>Learned predictors (shared native substrate, matched training set)</i>							
GAT-N	0.317	[0.254, 0.381]	-0.236 [-0.342, -0.125]	0.111	0.298	0.328	Yes
GAT-N-QoS	0.604	[0.538, 0.665]	+0.051 [-0.067, +0.169]	0.112	0.024	0.431	Yes
HGT	0.551	[0.474, 0.617]	-0.002 [-0.066, +0.072]	0.124	0.114	0.427	Yes
HGT-QoS	0.638	[0.561, 0.710]	+0.085 [-0.029, +0.194]	0.133	0.052	0.424	Yes
<i>Diagnostic reference — not a ranking model</i>							
RM / Q(v)	0.205	[0.092, 0.320]	-0.348 [-0.432, -0.265]	0.195	—	0.322	No

Label-noise ceiling. No predictor can exceed the reproducibility of its own labels. Re-running the oracle across the five seeds gives a test-retest rank correlation of 0.811–1.000 (median 0.982), so HGT-QoS’s $\rho = 0.638$ recovers roughly 65% of the attainable signal. Top- K sets are far noisier — cross-seed Jaccard median 0.847, falling to 0.370 on Logistics Fleet — which is why the Overlap@ K margins are less stable than the ranking ones. The least reproducible fold (Microservices, 0.811) is not one the typed model loses; on this corpus the low-ceiling folds and the lost folds are disjoint.

Key Insights concerning RQ1:

1. **SaG’s QoS-aware projection is the largest single gain.** Re-weighting shortest paths by declared QoS contracts lifts closed-form ranking from $\rho = 0.349$ to 0.553 and improves every one of the twelve held-out architectures (+0.204, $p = 0.0005$). Learned engines without the QoS channel reach only about this level (capacity-matched untyped GAT 0.563, HGT 0.551; Section 7.2), so the QoS-aware representation carries much of the signal.
2. **The learned engine leads, and matches the closed-form engine.** HGT-QoS has the highest mean correlation in Table 8 ($\rho = 0.638$) and wins 9 of 12 folds against Topo-QoS (+0.085, CI [-0.029, +0.194]). The registered confirmatory contrast does not reach significance ($p = 0.151$, Holm 0.303; un-augmented HGT: -0.002, $p = 0.470$), so we treat the two engines as comparable on this corpus. Section 7.5 evaluates a hybrid that combines them.
3. **Learned engines identify critical sets best.** On Overlap@ K , HGT-QoS (0.424), HGT (0.427) and GAT-N-QoS (0.431) all lead Topo-QoS (0.388) and Topo (0.366), although the fold-level differences are not significant ($\Delta = +0.037$, $p = 0.470$ for HGT-QoS).
4. **Where the engines differ.** HGT-QoS wins most clearly where the closed-form engine is weakest (Microservices +0.229, ATM +0.210) and loses where it is strongest (Enterprise -0.335, Telecom RAN -0.169; Section 7.2.1). The two engines are therefore complementary, which motivates the hybrid of Section 7.5.
5. **The explanation layer is an attribution instrument.** RM/ $Q(v)$ is listed for reference ($\rho = 0.205$, interval above zero); its role is to explain flagged components, not to rank them (Section 5).

7.1.2. The Active Stratum: Ranking Quality Separated From Inertness Detection

Between 21% (Microservices) and 52% (Healthcare) of each held-out Application population carries exactly zero simulated impact, so a predictor can score well by separating components that can propagate a failure from those that cannot, without ordering the propagating ones correctly. These are different capabilities, so we re-score all twelve folds on the *active stratum* — the $n_{>0}$ components with strictly positive impact — using the same predictions, folds and seeds. Table 9 reports both.

Two consequences follow.

1. **The restriction costs every predictor roughly half its correlation, learned or not.** Retained fractions run from 49% (RM) to 56% (HGT-QoS) with no systematic separation between the training-

Table 9: LOSO means over twelve folds on the full Application population (ρ) and restricted to components with strictly positive ground-truth impact ($\rho_{>0}$). Same predictions, folds, and seeds; only the evaluated subset differs. Retained is defined as the active-stratum ratio $\rho_{>0}/\rho$ (percentage of full-population correlation preserved).

Predictor	ρ (full)	$\rho_{>0}$ (active)	Retained
RM / $Q(v)$	0.205	0.102	49%
Topo	0.349	0.181	52%
Topo-QoS	0.553	0.280	51%
GAT-N	0.317	0.159	50%
GAT-N-QoS	0.604	0.328	54%
HGT	0.551	0.299	54%
HGT-QoS	0.638	0.356	56%

free and learned families (Topo-QoS 51%, GAT-N 50%, HGT 54%). Roughly half of every predictor’s full-population correlation reflects separating inert components from active ones, a property of the label distribution rather than a discriminator between methods.

2. **The method ordering is unchanged.** On the active stratum HGT-QoS still leads numerically ($\rho_{>0} = 0.356$), ahead of GAT-N-QoS (0.328) and Topo-QoS (0.280). No significance test is reported on this stratum, so it supports the full-population ordering descriptively and adds no new verdict.

7.2. RQ2: What the Learned Engine Needs

RQ2 Summary (Typing vs. QoS):. With model capacity and edge-channel width matched, the QoS edge channel is what improves learned ranking (+0.073 main effect, 10 of 12 folds; +0.072 for untyped models, $p = 0.016$), and relation-specific weights add nothing beyond it (typing main effect -0.014 , interaction +0.001). A capacity-matched untyped GAT with the QoS channel ($\rho = 0.635$) performs as well as HGT-QoS (0.622). The large typing gains of the unmatched comparison (+0.234) came from a $15\times$ capacity gap, not from typing.

Table 10 gives the 2×2 over relation typing (T) and the QoS edge channel (Q) for the four reported learned arms. The untyped arms there have 28,168 parameters against 434,620 for HGT, and read a 1-dimensional edge channel against HGT-QoS’s 16 dimensions. The control arms registered in Amendment 2 remove both differences. GAT-N-C and GAT-N-QoS16-C are untyped GATs widened to HGT’s parameter budget (437,496 and 429,992), and the latter reads the same 16-D edge channel as HGT-QoS, including the relation one-hot as an edge feature. All four matched arms ran in one CPU sweep (Table 11), and the decision rule was fixed before any control result existed.

Table 10: *Naive* 2×2 over relation typing (T) and the QoS edge channel (Q), unmatched in capacity ($15.4\times$) and edge-channel width; superseded for all typing claims by Table 11. Cells are the four reported learned arms: GAT-N ($\neg T \neg Q$), HGT (T-Q), GAT-N-QoS ($\neg T Q$), HGT-QoS (TQ). **Holm correction is applied across the three orthogonal quantities in the upper block only**; the four simple effects below are algebraically determined by those three and are reported descriptively (Section 7.3.1). Main effects average over the other factor’s levels; the typing effect reflects the joint transition to HGT and is confounded (Section 8.4). **Won** counts folds with $\Delta > 0$; the interaction is *negative* on all twelve. All quantities are post-hoc, and none were registered in advance. Fold overlap makes the reported p -values nominal (Section 8.3).

Quantity	Contrast	$\Delta\rho$	Won	W	p	p_{Holm}
<i>The 2×2: three orthogonal quantities, Holm-corrected across these three</i>						
Typing (main effect)	averaged over Q	+0.134	12/12	0.0	0.0005	0.0015
QoS channel (main effect)	averaged over T	+0.187	11/12	2.0	0.0015	0.0015
Typing $\times$ QoS interaction	difference of differences	−0.199	0/12	0.0	0.0005	0.0015
<i>Simple effects — descriptive, not separately corrected</i>						
Typing, QoS absent	HGT vs. GAT-N	+0.234	12/12	0.0	0.0005	—
Typing, QoS present	HGT-QoS vs. GAT-N-QoS	+0.035	9/12	19.0	0.1294	—
QoS channel, typing absent	GAT-N-QoS vs. GAT-N	+0.287	11/12	1.0	0.0010	—
QoS channel, typing present	HGT-QoS vs. HGT	+0.087	10/12	22.0	0.2036	—

Key insights concerning RQ2:

Table 11: The 2×2 with capacity and edge-channel width matched (PREREGISTRATION Amendment 2): **GAT-N-C** ($\neg T \neg Q$, 437,496 parameters), **HGT** ($T \neg Q$, 434,620), **GAT-N-QoS16-C** ($\neg T Q$, 429,992), **HGT-QoS** ($T Q$, 434,620). One CPU sweep, twelve LOSO folds, five seeds, Application population. Holm correction across the three orthogonal quantities; simple effects are descriptive. Cell means: 0.563, 0.548, 0.635, 0.622.

Quantity	Contrast	$\Delta\rho$	95% CI	Won	W	p	p_{Holm}
<i>Three orthogonal quantities, Holm-corrected across these three</i>							
Typing (main effect)	averaged over Q	-0.014	[-0.052, +0.023]	4/12	29.0	0.470	0.940
QoS channel (main effect)	averaged over T	+0.073	[+0.013, +0.120]	10/12	13.0	0.043	0.127
Typing $\times$ QoS interaction	difference of differences	+0.001	[-0.050, +0.042]	6/12	34.0	0.733	0.940
<i>Simple effects — descriptive, not separately corrected</i>							
Typing, QoS absent	HGT vs. GAT-N-C	-0.015	[-0.064, +0.033]	5/12	31.0	0.569	—
Typing, QoS present	HGT-QoS vs. GAT-N-QoS16-C	-0.013	[-0.054, +0.026]	4/12	27.0	0.380	—
QoS channel, typing absent	GAT-N-QoS16-C vs. GAT-N-C	+0.072	[+0.028, +0.109]	10/12	9.0	0.016	—
QoS channel, typing present	HGT-QoS vs. HGT	+0.073	[-0.002, +0.136]	10/12	19.0	0.129	—

- The QoS edge channel is the learned engine’s working ingredient.** At matched capacity, adding the 16-D QoS channel raises ranking by +0.07 with or without typing, on 10 of 12 folds each time, and significantly for the untyped pair (+0.072, CI [+0.028, +0.109], $p = 0.016$). The channel also stabilizes training: the seed spread of the untyped model falls from 0.083 to 0.010.
- Relation-specific weights add nothing once capacity is matched.** HGT and HGT-QoS are within 0.015 of their capacity-matched untyped counterparts and win only 4–5 of 12 folds against them. Because **GAT-N-QoS16-C** receives each edge’s relation type as a feature, the result is specifically that relation-typed *parameters* add nothing beyond relation-typed *inputs*. Message directionality (the HGT-QoS-U control) remains unmatched.
- The unmatched comparison overstated typing.** The 28,168-parameter **GAT-N** of Table 10 reaches $\rho = 0.317$; at HGT’s capacity the same untyped design reaches 0.563. Its +0.234 typing gain and -0.199 interaction are therefore effects of capacity and channel width. Table 10 is kept as the naive comparison it is, and it is not used for any claim about typing.

Neither the Fisher- z transform nor robust seed aggregation (Supplementary Sections S14 and S21), which left the unmatched interaction intact, could detect this confound: both hold the four unmatched arms fixed.

7.2.1. Where the Two SaG Engines Differ

HGT-QoS wins nine of twelve folds against **Topo-QoS**. Of its three losses, two are substantive — Enterprise ($\rho = 0.461$ vs. 0.795) and Telecom RAN (0.407 vs. 0.576) — and AV System is a near-tie (-0.030). All three losses fall where **Topo-QoS** is at its own strongest. Enterprise, AV and Telecom RAN are its 2nd, 3rd and 7th best folds of twelve (0.795, 0.753, 0.576), each at or above its mean of 0.553. Conversely, on the two folds where **Topo-QoS** is weakest (Microservices 0.265, ATM 0.311) the learned engine wins by +0.229 and +0.210. The engines are thus complementary: the learned model gains most where closed-form structure is least informative, and gives up ground where it is most informative. Section 7.5 exploits this directly.

Two properties of Enterprise may explain its deficit: it is the largest graph (520 nodes), so three rounds of message passing cover less of its diameter, and its derived projection is by far the densest in the corpus (26,276 edges, Table 16). Neither graph size ($\rho = -0.434$ with the margin, $p = 0.159$), connection density, nor prediction dispersion predicts in advance which engine will win on an unseen architecture.

7.3. RQ3: Ablations and Sensitivity Analysis

RQ3 Summary (Ablations and Sensitivity):. The reported orderings are robust. No configuration of the topic-weight or QoS sub-weight constants changes any comparison, only two of ten declared constants matter under Morris screening, and the behavioral queue-flow oracle agrees substantially with the cascade oracle ($\rho = 0.627$), which supports $I^(v)$ as a valid ranking target.*

This section presents ablations relevant to the primary claims, including the QoS edge encoding, cross-oracle agreement, and per-type stratification that informs the interpretation of the results that follow. Of the ten constants, only the AHP shrinkage λ and the Fault-Tolerance/Availability blend r_{FT} exhibit appreciable influence on ρ ($\mu^* = 0.134$ and 0.132 under Morris screening, compared to ≤ 0.025 for the remaining eight).

No configuration of the topic-weight or QoS sub-weight constants alters any comparison reported above. For the explanation layer, a uniform intra-dimension prior ranks better than the elicited AHP weights (0.319 vs. 0.200) and is recommended when $Q(v)$ is used to rank (Supplementary Section S1).

7.3.1. QoS Feature Ablation

To isolate the contribution of the continuous-categorical QoS edge features (Section 4.1.1) we evaluated **HGT**, an ablation whose edge features carry a constant unit weight and the relation one-hot. In the main sweep, **HGT-QoS** leads **HGT** by +0.087 (10/12 folds, $p = 0.204$). In the capacity-matched design the QoS channel adds +0.073 to typed and +0.072 to untyped models, each on 10 of 12 folds, with no interaction (Table 11). This is a consistent gain of about 0.07 regardless of architecture. The larger +0.287 of the unmatched untyped pair mostly reflects the small model’s instability, which the channel repairs.

The encodings also stabilise optimization, and there the asymmetry runs the other way: the median within-fold standard deviation over five seeds is 0.024 for **GAT-N-QoS** against 0.298 for **GAT-N**, and 0.052 for **HGT-QoS** against 0.114 for **HGT**. At matched capacity the same holds: 0.083 for **GAT-N-C** against 0.010 for **GAT-N-QoS16-C**. The QoS channel is the most reliable stabilizer of learned training.

How much QoS the target expresses.. These gains are earned against $I^*(v)$, whose ordering a topology-only relabeling recovers at mean $\rho = 0.965$ across the same twelve folds with no QoS term in the labeler (Section 4.3). On this target the QoS edge channel acts mainly as a relation-identity signal; QoS changes the label chiefly at its top- K boundary (Jaccard 0.678 between labels with and without QoS). Oracles that express QoS-driven impact in their ordering — deadline misses, durability replay, priority inversion — would let the encodings contribute contract semantics as well.

QoS Parameter Variance. Modal QoS shares range from 29% to 89% across the twelve scenarios, so every fold carries genuine variation in declared reliability, durability, and priority. As noted in Section 4.1.1, one schema dimension (`max_blocking_ms_log`) remains zero throughout the corpus as a reserved extension point, while the declared deadline populates the other two (`has_deadline`, `deadline_ns_log`) on 75% of topics. Reported gains therefore stem from six active dimensions.

7.3.2. Convergent Validity Over Simulation Oracles

The three reliability-facing oracles measure distinct constructs, so we checked whether they agree before treating any as ground truth. Over the twelve inductive folds on the Application population, the behavioural queue-flow oracle and the topological cascade injector agree at mean Spearman $\rho = 0.627$ (top- K Jaccard 0.370 against 0.111 expected by chance), against I^* ’s own seed-to-seed test-retest of 0.811–1.000. The agreement is therefore substantial but distinctly below label noise, which is the reading we want: an oracle reproducing another to within its own reproducibility would be re-measuring the same topology rather than corroborating it. Two boundaries qualify this — a large share of the agreement is the two oracles concurring on which components are *harmless*, and I_{dyn} has a measured noise floor of its own that the headline does not correct for.

The Four Golden Signals captured during execution show the mechanism behind the separation: crashing a critical publisher degrades delivery to surviving consumers ($I_{\text{dyn}} > 0$) while *reducing* their queue waits through contention relief ($\rho = -0.499$ against tail-latency delta). Since I^* is a deterministic breadth-first reachability computation and I_{dyn} a stochastic queue simulation, their moderate agreement is convergent validity between two independent formulations rather than re-measurement, and it places the learned predictors as surrogates for topological cascade reach, not detectors of queue dynamics (Section 8.3).

7.3.3. Node-Type Stratification and Attention

One result governs how every other number in this paper is read. Measured against $I_{\text{comp}}(v)$ over the twelve scenarios of the corpus, stratified RM rank correlations are $\rho = 0.597$ (Application), 0.317 (Broker), and 0.138 (Execution Host), while pooling all types gives $\rho = 0.217$ — less than two fifths of the Application figure it is supposed to summarise. This is why every evaluation is reported on a single stratum, and why pooled figures should not be read as summaries of any one entity type. Pooled correlation sits above the

Execution Host stratum (0.138), so this is aggregation bias rather than a strict Simpson reversal. A global sensitivity sweep of I_{comp} 's four severity weights ($N = 1,000$ Dirichlet draws; Supplementary Section S1.2) shows Application correlation exceeding pooled on *all* draws ($\rho \in [0.441, 0.599]$ vs. $[0.096, 0.351]$), while the strict condition holds on only 13.9% of the simplex. The stratification argument rests on the former, which is weight-invariant. A rule-based anti-pattern catalog on the same benchmark flags 93.4% of scored components and so does not discriminate; critical-set identification is delegated to the continuous rankers of Sections 7.1–7.2.

Aggregated by relation type over the ATM case study, first-layer mean HGT attention orders **USES** into libraries (0.227) above publish–subscribe channels (0.163–0.176). However, the spread across all seven relation types is narrow (0.15–0.23) and driven substantially by destination in-degree artifacts. Typed attention remains active across relation types without establishing a statistically distinct ordering.

7.4. RQ4: Zero-Shot Transfer to Models of Open-Source Systems

RQ4 Summary (Out-of-Generator Transfer):. Learned engines trained only on synthetic scenarios transfer to systems they have never seen. On five independently authored models of open-source systems, HGT-QoS reaches $\rho = 0.760$ [0.714, 0.819] and the capacity-matched untyped GAT with the same QoS channel reaches 0.805 [0.759, 0.868], against 0.51–0.53 for every training-free score. Both nearly double top- K critical-set overlap. On components that actually propagate failures, both learned engines keep positive correlation where every training-free score turns negative; with five systems these active-stratum differences remain unresolved.

We evaluated the framework on hand-authored architecture models of five open-source systems: Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and meshes modelled after Online Boutique and Train-Ticket (Section 6.1). They were written independently of the scenario generator, which is what makes them a transfer test, but they are models rather than extractions, and all carry labels from the same simulation oracles used throughout. The test is therefore transfer to independently authored topologies under simulated reachability, not agreement with field incident telemetry.

Two evaluations are conducted. The first is an exploratory evaluation of the closed-form explanation layer $Q(v)$ against the composite oracle $I_{\text{comp}}(v)$ ($\rho = 0.514$ –0.800); as noted in Section 4.3, I_{comp} 's four severity weights are unswept heuristics. The second, Section 7.4.1, evaluates learned predictors zero-shot against $I^*(v)$ (where RM achieves a lower mean $\rho = 0.516$). **Topo-QoS** is scored here on the flow projection, as everywhere else, because on the raw multigraph Application betweenness vanishes (Section 6.2.1). Standard middleware default contract profiles (e.g., ROS 2 Best-Effort/Reliable, MQTT QoS 0/1) were applied uniformly across edges lacking explicit manifests to ensure identical graph representations across baselines.

7.4.1. Zero-Shot Transfer of the Learned Model

To assess generalization outside the generator, HGT-QoS was trained on all twelve synthetic scenarios and evaluated zero-shot across five open-source systems versus $I^*(v)$ (five seeds; no open-source graph contributed training gradients or checkpoint selection). Tables 12–13 report this arm at the 3-layer, 300-epoch budget used for every other learned result in this paper. An earlier configuration used 2 layers and 150 epochs, chosen to limit over-smoothing across the smaller diameters of these meshes ($|V_{\text{app}}| \leq 41$); that reasoning appeals to a property of the evaluation targets, so although no target label or gradient reached the model, the configuration was not blind to the test systems and we do not report it as the primary result. It is uniformly slightly stronger ($\rho = 0.792$ against 0.760; $\rho_{>0} = +0.281$ against +0.236; $\text{Overlap}@K = 0.533$ against 0.470) and leaves every qualitative conclusion unchanged, which is the sensitivity we draw from it.

Key insights for out-of-generator transfer:

1. **Strong full-population transfer.** On all Applications, HGT-QoS reaches $\rho = 0.760$ [0.714, 0.819] against 0.511 (Topo), 0.526 (Topo-QoS) and 0.516 (RM), leading on 4 of 5 systems with a non-overlapping interval. Between 29% and 66% of Applications carry zero simulated impact, so part of this reflects correctly separating inert from active components — itself a useful triage property.
2. **Transfer does not depend on relation typing.** Trained and scored under the same zero-shot protocol, the capacity-matched untyped GAT with the 16-D QoS channel (**GAT-N-QoS16-C**) outperforms HGT-QoS on all five systems ($\rho = 0.805$ [0.759, 0.868] vs. 0.760; mean paired difference -0.045 for

Table 12: Zero-shot transfer to hand-authored models of five open-source systems, scored against $I^*(v)$ on the Application population under the protocol used throughout this paper (3 layers, 300 epochs; five seeds, $\pm$ = spread over seeds). No open-source graph contributed gradients or checkpoint selection. Training-free references are deterministic and are scored on identical labels, populations and node sets. Systems are grouped by the communication paradigm of the *original* system. All five models, including the two in the second group, are encoded as publish–subscribe graphs (Section 6.1). Where a system declares no explicit QoS manifest, default middleware contract profiles (ROS 2 Best-Effort/Reliable, MQTT QoS 0/1) are applied uniformly across baselines. Identification metrics for the same runs are in Supplementary Table S13.

System model	$ V_{\text{app}} $	$n_{>0}$	RM (ρ)	Topo (ρ)	Topo-QoS (ρ)	HGT-QoS (ρ)	HGT-QoS ($\rho_{>0}$)
<i>Originals are publish–subscribe systems</i>							
Autoware.universe (ROS 2)	32	19	0.357	0.307	0.378	0.716 $\pm$ 0.081	+0.517
EdgeX Foundry (Industrial IoT)	22	10	0.470	0.534	0.534	0.793 $\pm$ 0.037	+0.183
Home Assistant (Smart Home)	24	17	0.265	0.297	0.289	0.864 $\pm$ 0.063	+0.702
<i>Originals are RPC systems (modelled as pub-sub)</i>							
Online Boutique (pub-sub model)	22	8	0.777	0.891	0.888	0.710 $\pm$ 0.070	−0.031
Train-Ticket Booking Mesh	41	14	0.713	0.528	0.541	0.717 $\pm$ 0.096	−0.192
Mean	—	—	0.516	0.511	0.526	0.760	+0.236

Table 13: Means over the five system models on the full Application population (ρ) and restricted to components with strictly positive ground-truth impact ($\rho_{>0}$), with percentile bootstrap intervals over the five systems ($B = 2,000$). All four predictors are scored on identical labels, populations and node sets from one run, so the columns are commensurable. At $n = 5$ the intervals are descriptive and carry no significance claim. **The full-population intervals separate the learned model from all three baselines; every active-stratum interval spans zero**, which is why RQ4 is reported as established on the full population and unresolved on the active one.

Predictor	ρ (full), 95% CI	$\rho_{>0}$ (active), 95% CI
RM / $Q(v)$	0.516 [0.343, 0.680]	−0.055 [−0.292, +0.213]
Topo	0.511 [0.346, 0.703]	−0.083 [−0.269, +0.108]
Topo-QoS	0.526 [0.357, 0.699]	−0.092 [−0.268, +0.094]
HGT-QoS	0.760 [0.714, 0.819]	+0.236 [−0.053, +0.525]

HGT-QoS). It also scores higher on $\text{Overlap}@K$ (0.519 vs. 0.470), PR-AUC (0.790 vs. 0.713) and active-stratum correlation (+0.319 [0.001, 0.638] vs. +0.236). Consistent with Section 7.2, what transfers is learning over SaG’s QoS-annotated graph, not relation-specific parameters.

- Active components.** Restricted to components that propagate failures, the learned model keeps a positive mean correlation ($\rho_{>0} = +0.236$) where every training-free score turns negative (−0.055 to −0.092). Its interval [−0.053, +0.525] spans zero, as do the baselines’, so this comparison is unresolved at five systems. $\rho_{>0}$ is positive on the three models of publish–subscribe systems (Home Assistant +0.702, Autoware +0.517, EdgeX +0.183) and non-positive on the two modelled after RPC systems (Online Boutique −0.031, Train-Ticket −0.192).
- Identification is where the learned engine separates most clearly.** On top- K overlap it averages 0.470 [0.410, 0.540] against 0.248 [0.09, 0.46] for the closed-form scores; on threshold-free measures the gap is wider ($F_1@7$ 0.473 vs. 0.29–0.33; PR-AUC 0.713 vs. 0.47–0.52; Supplementary Table S13). On EdgeX, symmetric adapter-to-broker star connections create betweenness ties that collapse closed-form triage entirely ($\text{Overlap}@K = 0.000$), while the learned engine still separates components.
- Scope of the 3–2 split.** Both RPC-derived models are encoded with topics and brokers and labelled by the same forward-reachability oracle, so the split cannot be attributed to call-tree semantics; testing that hypothesis requires synchronous edges in the schema and a backward-propagating oracle (Section 8.4). On the Online Boutique model the closed-form scores lead only on the full population (Topo-QoS 0.888) and are negative on its active components (−0.072).

7.5. SaG-Hybrid: Learning a Correction to the Closed-Form Engine

Section 7.2.1 showed that the two SaG engines are complementary: the learned engine gains most where closed-form structure is least informative, and gives up ground where it is most informative. SaG-Hybrid combines them directly. It is HGT-QoS with one extra input per Application and Library — the closed-form Topo-QoS score, rank-normalized within the graph — and an output that adds a learned correction to that score on the logit scale, $\hat{I}^*(v) = \sigma(z(v) + \alpha \text{logit}(p(v)))$, with a single learnable α . Everything else

(architecture, loss, epochs, early stopping, seeds) is identical to HGT-QoS, and it adds 321 parameters (434,941 in all). The design, the contrasts and a decision rule were registered before any run (Amendment 5), and no setting was tuned.

The hybrid was evaluated in one CPU sweep, with the comparators re-run in the same invocation and never mixed with the GPU rows of Table 8. HGT-QoS reaches $\rho = 0.622$ on CPU against 0.638 on GPU, within the run-to-run displacement of Section 8.3; the training-free rows are identical.

Table 14: SaG-Hybrid under the LOSO protocol of Table 8 (twelve folds, five seeds, Application population), all rows from one CPU sweep, and zero-shot on the five open-source system models under the protocol of Table 12. $\Delta\rho$ is paired by fold against Topo-QoS, with a bootstrap 95% CI and a two-sided Wilcoxon test. Holm correction is across the two registered hybrid contrasts (vs. Topo-QoS and vs. HGT-QoS).

Predictor	LOSO, twelve synthetic architectures					Five system models	
	Mean ρ [95% CI]	$\Delta\rho$ vs Topo-QoS [95% CI]	Won	p (p_{Holm})	Overlap@K	ρ [95% CI]	PR-AUC
Topo	0.349 [0.254, 0.452]	-0.204 [-0.286, -0.122]	0/12	0.0005	0.366	0.511 [0.346, 0.703]	0.474
Topo-QoS	0.553 [0.443, 0.657]	—	—	—	0.388	0.526 [0.357, 0.699]	0.474
HGT-QoS	0.622 [0.547, 0.690]	+0.069 [-0.046, +0.174]	8/12	0.266	0.426	0.760 [0.714, 0.819]	0.713
SaG-Hybrid	0.657 [0.572, 0.733]	+0.103 [+0.055, +0.152]	11/12	0.0034 (0.0068)	0.435	0.695 [0.643, 0.730]	0.602

Key insights for SaG-Hybrid:

- The hybrid significantly outperforms the closed-form engine out of distribution.** SaG-Hybrid reaches $\rho = 0.657$ against 0.553 for Topo-QoS (+0.103, CI [+0.055, +0.152]), winning 11 of 12 held-out architectures ($W = 4.0$, $p = 0.0034$, Holm 0.0068). This meets the decision rule registered before the run, so it is the headline learned engine of this paper. It also has the highest Overlap@K (0.435) and active-stratum correlation ($\rho_{>0} = 0.340$) under LOSO.
- It removes the learned engine’s failure mode.** The folds that held HGT-QoS back were those where closed-form structure is most informative. With the prior, the Enterprise deficit against Topo-QoS shrinks from -0.369 to -0.061 , and Telecom RAN turns from -0.149 into a win (+0.071). Enterprise is the hybrid’s only loss.
- Hybrid vs. pure learning is a trade-off.** Against HGT-QoS the hybrid is +0.035 on average (8/12, $p = 0.733$): it gains on the closed-form engine’s strongest folds and gives some ground on its weakest (Healthcare, IoT Smart City, ATM, Microservices), where the prior is least informative.
- Pure learned engines transfer further.** On the five independently authored system models, SaG-Hybrid ($\rho = 0.695$) stays well above every training-free score (0.51–0.53) but below HGT-QoS (0.760) and the capacity-matched untyped GAT with the QoS channel (0.805; Section 7.4). A hybrid built on that untyped engine has not been evaluated.

7.6. RQ5: Analysis Cost and Its Comparison Against Simulation

RQ5 Summary (Analysis Cost):. Neural inference is effectively free (56 ms for a 2,000-node architecture, 0.02% of pipeline time). Cost is dominated by deterministic feature extraction, specifically the $O(|V|^2 + |V||E|)$ Connectivity Degradation Index, and tracks the density of the derived dependency projection rather than component count. Cold extraction takes 0.16–79.3 s per scenario, 2–18 $\times$ (median 5.6 $\times$) the in-process cascade simulation, so SaG’s advantage is avoiding staging infrastructure rather than CPU time.

RQ5 quantifies computing overhead and sustainability during CI/CD evaluation, with per-stage latencies reported in Table 15:

The neural stage is the cheapest by a wide margin and the deterministic one is not. At 2,000 components the HGT forward pass takes 56 ms against 239 s for structural analysis, a ratio of 4,259 $\times$ — but that is a ratio between pipeline stages, not a cost of evaluation, because indices 0–17 of every node feature vector are produced by the analysis stage the forward pass depends on. End-to-end evaluation of an unseen 2,000-component architecture takes about four minutes, of which the learned model is 0.02%; the 56 ms is the marginal cost of re-scoring an already-analysed graph. Across the corpus the complete gate (structural analysis plus 18 anti-pattern detectors, whose share is ≤ 0.19 s) executes in 0.16–79.3 s (Table 16).

One metric dominates cost. Across the endpoints of Table 15 the measured cost is consistent with the stage’s $O(|V|^2 + |V||E|)$ bound: from 249 to 1,998 components wall-clock rises 138 $\times$ against a 137 $\times$

Table 15: Per-stage latency of the inference pipeline across scaling graph sizes (CPU, median of 3 runs; 5 for the forward pass). The analysis stage is stable across repeats (p10–p90 within 1% of the median everywhere) while the forward pass is not, which is why its column carries a spread: at 56 ms the measurement is dominated by interpreter and dispatch overhead rather than by the graph. The 249-node row is slower than the 499-node row for the same reason: it is measured first, and its median still carries first-call allocation and dispatch warm-up.

$ V $	$ E $	Analyze (s)	Graph → Tensor (s)	HGT Forward (ms)	Analyze : Forward	Forward p10–p90 (ms)
249	1,127	1.74	0.010	26.5	66×	13.0–34.4
499	2,402	8.32	0.022	16.4	509×	15.6–16.4
999	6,422	44.54	0.056	21.1	2,108×	19.1–36.5
1,998	19,301	239.34	0.157	56.2	4,259×	43.8–57.8

growth in $|V||E|$. That is an endpoint coincidence rather than a tracked curve: between the middle rows the component count doubles ($499 \rightarrow 999$) while wall-clock rises $5.4\times$, faster than the bound requires, and the series varies $|V|$ and $|E|$ together so it cannot separate them — Table 16 does that on the corpus. The dominant term is the Connectivity Degradation Index, computed for every node in the main connected component rather than for articulation points alone. That is a correctness requirement, not an oversight: gating CDI to articulation points leaves it identically zero wherever removal does not literally disconnect the graph, driving $A(v)$ to a near-constant in exactly the redundant multi-publisher topologies this system targets. The cost is the price of a non-degenerate Availability score.

Table 16: Static analysis gate against the cascade-reachability oracle, per scenario, from one paired measurement session. **Gate** is structural analysis plus the 18 anti-pattern detectors, whose own share is negligible (≤ 0.19 s everywhere). **Oracle** is the full five-seed ground-truth labelling sweep over Application, Broker and Library nodes. $|E_{\text{proj}}|$ is the number of derived **DEPENDS_ON** edges in the Application–Library projection the analysis stage traverses, read from the committed cache. The ratio is $2.0\text{--}17.7\times$ with a median of $5.6\times$: the eighteen-fold figure quoted elsewhere in this section is the maximum, not the typical case. Ordering by $|E_{\text{proj}}|$ rather than by $|V|$ is what makes the column monotone — Enterprise carries 300 applications over only 120 topics, so its Rule-1 projection is near-complete, while IoT Smart City has more components and a sixth of the edges at a third of the cost.

Scenario	$ E_{\text{proj}} $	Gate (s)	Oracle (s)	Ratio
Enterprise	26,276	79.27	4.487	$17.7\times$
Enterprise Integration (ESB)	4,641	3.17	0.245	$12.9\times$
AV System	3,073	2.61	0.355	$7.4\times$
Financial Trading	2,657	1.71	0.239	$7.1\times$
Real-Time Gaming	2,318	2.43	0.379	$6.4\times$
Healthcare	1,590	0.92	0.149	$6.2\times$
IoT Smart City	1,835	5.48	1.104	$5.0\times$
Telecom RAN	2,116	3.70	1.065	$3.5\times$
Logistics Fleet	1,660	2.74	0.939	$2.9\times$
Microservices (synthetic)	1,524	2.29	0.828	$2.8\times$
Industrial SCADA	897	2.43	1.191	$2.0\times$
ATM System	155	0.16	0.080	$2.0\times$
Median	—	—	—	$5.6\times$

7.6.1. Comparison With Direct Simulation

Timing the cascade reachability labeling sweep (five seeds, node types Application/Broker/Library, the full ground-truth run) over all twelve scenarios, on the same machine, at the same commit and in the same measurement session as the gate, gives $0.08\text{--}4.49$ s per scenario against $0.16\text{--}79.3$ s for the analysis gate. Both maxima occur in the 520-component Enterprise mesh, so the largest scenario compares 4.5 s of simulation against 79.3 s of static analysis: there the gate costs roughly eighteen times as much as the simulation. Pairing the sweeps scenario by scenario (Table 16) gives $2.0\text{--}17.7\times$, median $5.6\times$. The typical premium is therefore far below the maximum, but the gate is more expensive on *all twelve* scenarios. What predicts the premium is the derived projection’s size, not the component count — the ratio correlates with $|E_{\text{proj}}|$ at $\rho = 0.95$ against 0.79 for cost against $|V|$. Enterprise is the outlier because its 300 applications share only 120 topics, so Rule 1 derives a near-complete graph of 26,276 edges, while IoT Smart City has more components,

an eighth of the edges and a third of the cost. These are wall-clock figures on one commodity CPU, read to one significant figure: across sessions the gate maximum ranges over 77–83 s and the oracle maximum over 4.5–4.8 s, giving $16.7\times$ and $17.7\times$ on two independently paired sessions. Both halves of Table 16 share a commit and corpus digest.

Breadth-first cascade traversal is cheaper than all-pairs connectivity degradation ($O(|V|^2 + |V||E|)$), so on raw CPU time direct simulation is faster wherever its parameters are available. SaG’s analysis additionally scores infrastructure components and dependency edges that node-level message-flow simulation leaves unscored, and its metrics are amenable to caching across commits so that only the k -hop neighbourhood of a change is recomputed (not implemented here; Table 15 times full recomputation).

Training cost. The figures above are inference costs. Training is a separate, one-off cost per model version: the last sweep with recorded per-arm wall-clock (CPU, sequential, 12 folds $\times$ 5 seeds = 60 fits per arm) took 0.6 h for GAT-N, 0.9 h for GAT-N-QoS, 1.4 h for HGT and 4.9 h for HGT-QoS, 7.7 CPU-hours in all. The GPU sweep behind Table 8 did not record per-fit durations.

8. Discussion, Threats to Validity, and Limitations

8.1. Discussion and Practical Consequences

The representation carries the signal. The most robust result of this study is that SaG’s QoS-aware dependency projection makes criticality legible to simple and learned analyzers alike. A closed-form centrality on the projection improves every held-out architecture over its unweighted form (+0.204). For learned engines, the QoS edge channel is the component that matters (+0.07 at matched capacity, Section 7.2), and letting a learned engine correct the closed-form score yields the best ranking on unseen synthetic architectures (Section 7.5). Practitioners therefore gain most from modeling their architecture with typed entities and declared QoS contracts, whichever engine they then run.

Choosing an engine.

- **Closed-form engine (Topo-QoS).** It needs no training or checkpoints, reaches $\rho = 0.553$ zero-shot across twelve synthetic architectures, and is the natural default for lightweight CI gates. On this reachability target it is on par with the learned engine within the synthetic corpus.
- **Learned engines (HGT-QoS, GAT-N-QoS16-C).** They give the best critical-set identification and the strongest transfer to independently authored systems ($\rho = 0.760$ and 0.805 vs. 0.51 – 0.53 ; PR-AUC 0.71 – 0.79 vs. about 0.5). They gain most on dense, irregular topologies where closed-form structure is least informative (Microservices +0.229, ATM +0.210 for HGT-QoS), and score a new architecture in milliseconds once its features exist. Because relation-specific weights add nothing at matched capacity, a sufficiently wide untyped GAT with the QoS channel is the simpler choice and transferred best in this study.
- **Hybrid engine (SaG-Hybrid).** The best engine on unseen synthetic architectures ($\rho = 0.657$) and the only one that significantly outperforms closed-form ranking (+0.103, 11/12 folds). Because it starts from the closed-form score, it keeps the closed-form engine’s strength on dense projections such as Enterprise while adding the learned engine’s gains elsewhere. It is the recommended default when an architecture resembles the training distribution; pure learned engines remain preferable for transfer to substantially different systems (0.76 – 0.81 vs. 0.695).
- **Explanation layer.** The RM profile (Section 5) names a remediation class for each flagged component — Availability-driven replication versus Fault-Tolerance-driven circuit breakers — while the engines set triage priority.

Architectural correlates of learned-engine performance. Table 17 collects the factors that co-vary with where the learned engine does well on this corpus. Each rests on one to three folds or systems, so they are working hypotheses for practitioners and future studies.

Table 17: Observed correlates of learned-engine performance on the seventeen evaluated architectures, with a candidate mechanism for each and the evidence on this corpus.

Factor	Candidate mechanism	Evidence on this corpus
QoS edge channel	Edge-level QoS contracts tell the model how strongly each dependency couples components.	+0.07 at matched capacity on 10/12 folds, typed or untyped; relation-specific weights add nothing (Section 7.2).
Topology and symmetry	Dense irregular meshes give distinct neighborhoods; symmetric stars create betweenness ties.	Microservices and ATM, +0.229 and +0.210 over Topo-QoS ; EdgeX, closed-form $\text{Overlap}@K = 0.000$.
Scale and diameter	Fixed 3-layer message passing covers less of a large graph.	520-node Enterprise: 0.461 vs. 0.795 (one scenario).
Original system's paradigm	Unknown; all five models are encoded as publish-subscribe graphs.	$\rho_{>0} > 0$ on the 3 pub-sub-derived models; non-positive on the 2 RPC-derived models.
Inert-node base rates	Zero-impact components are part of what full-population correlation rewards.	21%–52% of applications carry $I^*(v) = 0$; $\rho_{>0}/\rho \approx 49\%$ –56%.

Table 18: How the instruments in the SaG portfolio are best used, given the evidence in Section 7.

Instrument	Context	Role and evidence
Topo-QoS (Closed-form)	Lightweight CI gates	Training-free, $\rho = 0.553$ out of distribution; +0.204 over unweighted centrality on 12/12 folds.
Learned + QoS channel	Substantially different or irregular architectures	Best transfer (GAT-N-QoS16-C 0.805, HGT-QoS 0.760) and identification (PR-AUC 0.71–0.79).
SaG-Hybrid	Default learned engine	Best LOSO ranking ($\rho = 0.657$); significantly above Topo-QoS (+0.103, 11/12 folds, Holm $p = 0.0068$).
RM explanation layer	Refactoring and root-cause discussion	ISO/IEC 25010 attribution (Availability vs. Fault Tolerance vs. Maintainability).

8.2. Performance and Computational Sustainability Implications

Green software engineering assesses energy across development, assurance, and execution [29, 91, 92, 93, 94, 30, 31]. Pre-deployment analysis avoids provisioning staging clusters for chaos sweeps; we state this as infrastructure avoidance rather than a measured energy saving. The computation SaG itself expends is small. At base SoC power (28 W), one pass of the analysis gate over all twelve scenarios costs at most 3.0 kJ (0.83 Wh), and neural inference is negligible. Training the four learned arms once (7.7 CPU-hours; Section 7.6.1) costs about 0.78 MJ (0.22 kWh) on the same bound and is amortized over every subsequent evaluation. These are upper bounds from wall-clock time; direct RAPL/NVML measurement [30, 32, 33] is future work. On raw CPU time, the in-process cascade simulation remains cheaper than cold feature extraction (2–18 $\times$, median 5.6 $\times$; Table 16), so incremental caching of structural metrics across commits is the main lever for further reducing analysis cost.

8.3. Threats to Validity

Construct validity. All labels are simulator-derived rather than observed failures. Two independent oracles support the primary target: the behavioral queue-flow simulation agrees with the cascade oracle at $\rho = 0.627$ (top- K Jaccard 0.27–0.37 across the three oracle pairs; Section 7.3.2). Because $I^*(v)$ is a reachability functional of the same topology the predictors read, a strong closed-form comparator is expected; retargeting the LOSO contrasts on I_{dyn} , which is not recoverable in closed form, is the next experiment. The five open-source system models were authored by one author from public documentation (Section 6.1); independent re-modeling, ideally extracted from deployment manifests, would strengthen RQ4.

Internal validity. Predictors consume G_{analysis} while oracles traverse $G_{\text{structural}}$, which is asserted in CI. Substrate, training set, depth and early stopping are matched across learned arms. The reported typed and untyped arms differ in capacity (434,620 vs. 28,168) and edge-channel width; the registered capacity- and channel-matched control (Table 11) removes both, and all typing conclusions rest on it. Message directionality is still unmatched (the **HGT-QoS-U** control has not been run). “QoS-off” arms still receive QoS through four centralities computed on the weighted projection (Section 3.4). Architectural hyperparameters follow conventional HGT values and were not tuned on any evaluation split.

External validity. The synthetic corpus spans eight operational domains from one generator family; the five system models add independently authored topologies of 22–41 applications. Scaling beyond 2,000 nodes would benefit from incremental caching or mini-batching [95].

Conclusion validity. We use Spearman ρ , bootstrap intervals over folds ($B = 2,000$) and Wilcoxon signed-rank tests. LOSO folds share training scenarios, so p -values are nominal [90] and are read alongside fold-level sign consistency. Every analysis is stratified by entity type (Section 7.3.3).

Repeatability. At fixed code, seeds and device, every reported figure reproduces at its reported precision: a clean re-run of the hybrid sweep from a tagged commit changed no cell by more than 1.3×10^{-4} , the residue of tie-breaking order in QoS-weighted betweenness. All 180 training-free cells of the main sweep also reproduce across devices. Learned cells move across code revisions and devices (up to 0.172 in fold mean between the last two sweeps; HGT-QoS moved least, 0.041), owing to a since-fixed PyTorch Geometric device-placement issue, stale checkpoint resumption, and non-deterministic CUDA reductions. Learned figures are therefore reported against the released artifacts (`reproduce/rerun_drift.py`), and comparisons are always made within one sweep: the SaG-Hybrid contrasts use comparators re-run in the same CPU invocation. Re-running the zero-shot HGT-QoS evaluation on CPU reproduced every published per-system value of Table 12.

8.4. Limitations and Future Work

The explanation layer’s attributions have not yet been evaluated with developers or against injected faults; a mutation benchmark and a practitioner study are planned. No published learned-criticality model (FINDER [67], DrBC [68]) has been reproduced on this corpus. Scoring hosts and network links, robustness to missing operational parameters, and incremental CI re-scoring are natural capabilities of the learned engine that this study does not yet evaluate.

Future directions. (1) Build SaG-Hybrid on the capacity-matched untyped QoS engine, and run the remaining directionality control (HGT-QoS-U); (2) retarget RQ1 and RQ2 on the queue-flow oracle I_{dyn} ; (3) reproduce a published learned-criticality baseline; (4) add synchronous call edges and a backward-propagating oracle, so that RPC and hybrid architectures can be modeled natively; (5) extract the open-source system models from real deployment manifests; (6) validate rankings against production incident data; (7) measure energy directly via RAPL/NVML.

9. Conclusion

This study showed that typed, QoS-aware architecture models support accurate pre-deployment ranking of components by cascading-failure impact, without runtime telemetry. Software-as-a-Graph turns architecture descriptions into typed multigraphs with QoS-weighted dependencies. Engines built on this representation were evaluated under leave-one-scenario-out cross-validation over twelve synthetic architectures and zero-shot on five independently authored models of open-source systems.

The representation carries much of the signal: QoS-aware projection raises closed-form ranking from $\rho = 0.349$ to 0.553 on every held-out architecture. For learned engines, a control matched in capacity and edge-channel width showed that SaG’s QoS edge encoding is the working ingredient (+0.07 on 10 of 12 folds), while relation-specific weights add nothing beyond it. Learned engines transfer well beyond their training distribution ($\rho = 0.76$ –0.81 against 0.51–0.53 on the system models, with roughly double the top- K critical-set overlap). Learned and closed-form reasoning are complementary. SaG-Hybrid, in which a learned engine corrects the closed-form score, is the best engine on unseen synthetic architectures ($\rho = 0.657$) and significantly outperforms closed-form ranking on 11 of 12 folds (+0.103, Holm $p = 0.0068$), as registered before the experiment. An ISO/IEC 25010 attribution layer complements the rankings by naming the remediation each flagged component calls for.

For practice, SaG offers a training-free closed-form engine for lightweight CI gates, the hybrid engine for the most accurate ranking on familiar architectures, and a QoS-aware learned engine for transfer to substantially different systems. The next steps are to build the hybrid on the untyped QoS-aware engine, to retarget the engines on the behavioral queue-flow oracle, to extract system models directly from deployment manifests, and to validate the rankings against production incident data.

Declarations

CRediT Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision. **Declaration of Competing Interest.** The authors declare no competing financial interests or personal relationships that could have influenced this work. **Funding.** This research received no external grant. **Data Availability.** The replication package (datasets, harnesses, checkpoints, scripts) is available on Zenodo under DOI 10.5281/zenodo.14922108 [96] with `uv/pip` environments. Synthetic datasets regenerate byte-identically. The deposit ships all artifacts backing reported tables as a dated bundle (`SaG_JSS_Results_<stamp>`) with a `MANIFEST.json` recording SHA-256 digests, commit hashes, and corpus provenance. Four supplementary artifacts predate provenance stamping and carry no commit or corpus digest: `atm_scale_sweep_v3.json` (S6), `qos_label_ablation.json` (Section 4.3), `threshold_sensitivity_v3.json` (S3) and `topic_weight_sensitivity_v3.json` (S1); their correspondence to the corpus is asserted by the bundle rather than recorded in the file. The verification script (`reproduce/reconcile_manuscript.py`) runs standalone against the deposit, mechanically verifying all 430 table quantities against JSON artifacts.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic’s Claude to assist with typesetting, LaTeX formatting, and the development of analysis and reporting scripts in the replication package. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors’ own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [14] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [15] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [16] W. Cunningham, The WyCash portfolio management system, in: *Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 1992, pp. 29–30.
- [17] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [18] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.

- [19] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [20] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering* SE-2 (4) (1976) 308–320.
- [21] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [22] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [23] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [24] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [25] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [26] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [27] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [28] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish–subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [29] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [30] L. Lannelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, *Advanced Science* 8 (12) (2021) 2100707. doi:10.1002/adv.202100707.
- [31] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [32] V. Schmidt, K. Goyal, A. Joshi, B. Feld, L. Conell, N. Laskaris, D. Sarthou, H. Verreault, J. Blank, S. Zhang, Codecarbon: Estimate and track carbon emissions from machine learning computing, *Journal of Open Source Software* (2021).
- [33] A. Noureddine, Joularjx: A java-based software power monitoring tool, *SoftwareX* 14 (2021) 100688.
- [34] R. C. Cheung, A user-oriented software reliability model, *IEEE Transactions on Software Engineering* SE-6 (2) (1980) 118–125.
- [35] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.
- [36] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, *Software and Systems Modeling* 7 (1) (2008) 49–65.
- [37] S. Becker, H. Koziolek, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [38] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, *IEEE Transactions on Software Engineering* 35 (2) (2009) 148–161.
- [39] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: *2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [40] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [41] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [42] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.
- [43] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: *Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS)*, 2021.
- [44] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2022.
- [45] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2023.
- [46] X. Meng, P. Shen, Y. Sun, D. Liu, J. Lu, S. Zhang, D. Pei, Microcause: Root cause analysis for microservice systems through graph neural networks, in: *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 403–414.
- [47] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3715005.
- [48] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [49] S. A. Bohner, R. S. Arnold, *Software Change Impact Analysis*, IEEE Computer Society Press, Los Alamitos, CA, 1996.
- [50] S. Esparrachiari, T. Reilly, A. Rentz, Tracking and controlling microservice dependencies, *ACM Queue* 16 (4) (2018). doi:10.1145/3277539.3277541.
- [51] X. Yang, K. Tang, X. Yao, A learning-to-rank approach to software defect prediction, *IEEE Transactions on Reliability* 64 (1) (2015) 234–246.
- [52] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
- [53] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 580–586.
- [54] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: *Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE)*, 2007.
- [55] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
- [56] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction,

- in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2021. doi: 10.1109/ASE51524.2021.9678749.
- [57] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, arXiv preprint (2024). arXiv:2412.08352, doi:10.48550/arXiv.2412.08352.
- [58] A. Santos, A. Cunha, N. Macedo, Statistical and model-driven static analysis of ROS systems, IEEE Transactions on Software Engineering 47 (10) (2019) 2200–2218.
- [59] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
- [60] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.
- [61] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.
- [62] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.
- [63] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.
- [64] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [65] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
- [66] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
- [67] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature Machine Intelligence 2 (2020) 317–324.
- [68] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.
- [69] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [70] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.
- [71] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.
- [72] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.
- [73] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.
- [74] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.
- [75] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
- [76] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.
- [77] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE), 2025.
- [78] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 9244–9255.
- [79] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 19620–19631.
- [80] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann, 1988.
- [81] G. Beliaikov, A. Pradera, T. Calvo, Aggregation functions: A guide for practitioners, Studies in Fuzziness and Soft Computing 221 (2007).
- [82] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, IEEE Transactions on Systems, Man, and Cybernetics 18 (1) (1988) 183–190.
- [83] G. H. Hardy, J. E. Littlewood, G. Pólya, Inequalities, 2nd Edition, Cambridge University Press, 1952.
- [84] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
- [85] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
- [86] Team SimPy, Simpy: Discrete event simulation for Python, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [87] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
- [88] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.
- [89] C. Spearman, The proof and measurement of association between two things, American Journal of Psychology 15 (1) (1904) 72–101.
- [90] C. Nadeau, Y. Bengio, Inference for the generalization error, Machine Learning 52 (2003) 239–281. doi:10.1023/A:1024068626366.
- [91] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, Communications of the ACM 63 (12) (2020) 54–63. doi:10.1145/3381831.
- [92] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), Florence, Italy, 2019, pp. 3645–3650. doi:10.18653/v1/P19-1355.
- [93] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, J. Dean, Carbon emissions and large neural network training, arXiv preprint arXiv:2104.10350 (2021). doi:10.48550/arXiv.2104.10350.

- [94] S. Georgiou, M. Kechagia, T. Sharma, F. Sarro, Y. Zou, Green AI: Do deep learning frameworks have different costs?, in: Proceedings of the 44th International Conference on Software Engineering (ICSE), 2022, pp. 1082–1094. doi:10.1145/3510003.3510221.
- [95] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, GraphSAINT: Graph sampling based inductive engine, in: Proc. International Conference on Learning Representations (ICLR), 2020.
- [96] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/zenodo.14922108.